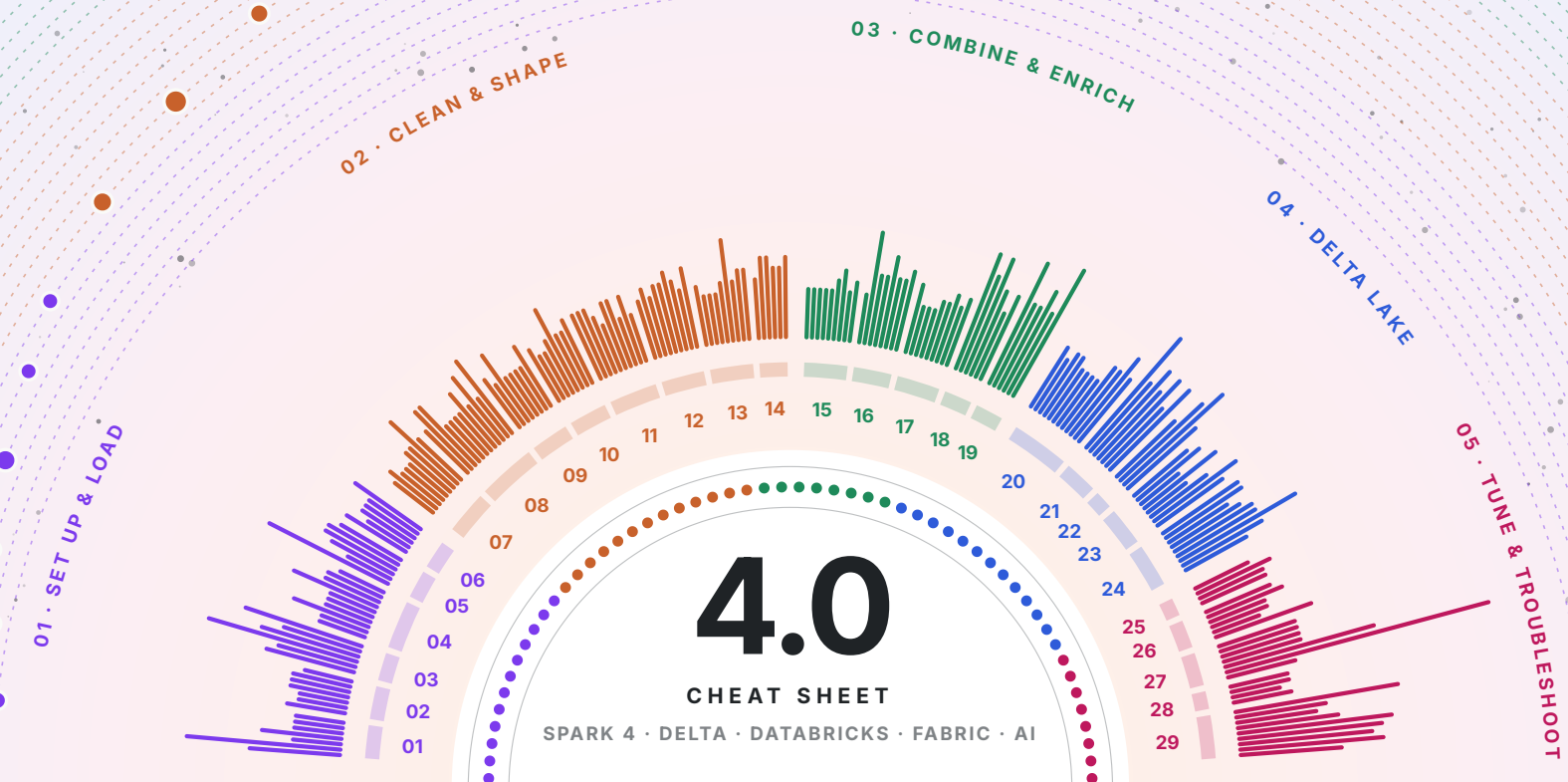


PySpark Code.

221 patterns for Databricks and Fabric, from a SparkSession to Delta, streaming and AI functions.



01 Set up & Load			02 Clean & Shape			03 Combine & Enrich			04 Delta Lake			05 Tune & Troubleshoot			
01	Setup & Import	04	07	Nulls & Duplicates	18	15	Joins	30	20	Delta & Time Travel	51	25	Spark Plans	62	
02	Config Settings	06	08	Filtering	19	16	Aggregations	36				26	Performance & Optimisation	64	
03	Notebook Utilities	09	09	Ordering	21	17	Window Functions	38	21	Delta: Advanced MERGE	53	27	Performance: Skew Handling	66	
			10	Column Operations	22				22	SCD Type 1	55				
04	I/O	11				18	UDFs	45	23	SCD Type 2	56	28	Debugging	69	
05	Schema	13	11	String Operations	23	19	AI Functions	46	24	Streaming	58	29	Error Handling & Logging	70	
06	Data Types	15	12	Date & Time	25										
			13	Complex Data Types	26										
			14	Loops	28										
Appendix A Common errors			73			Appendix B Spark 3.5 vs 4.x			74			Notes What changed since 3.0			76

BY

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn

COMPANION

The PySpark Practitioner's Handbook 4.0

97 concepts on how Spark thinks and runs, each with a figure.



WHY THIS EXISTS

Keep it open while you code.

Learning PySpark rarely stalls for lack of material. It stalls because the material is everywhere: the API docs in one tab, a Databricks blog in another, a Fabric page, and a forum answer from years ago that no longer works on Spark 4. This sheet puts the patterns you reach for every day in one place, on one dataset, so you spend your time writing code instead of hunting for it.

01 The struggle is the point

The first weeks feel slow. Lazy evaluation, shuffles, a join that runs for twenty minutes. That friction is how the way Spark thinks gets into your head. Stay with it, and type the code yourself.

02 Keep it beside your notebook

It is built to sit next to your editor, not on a shelf. Find the task, adapt the pattern to your own columns, run it, and compare what you get with what the sheet shows.

03 Write it first, then ask the AI

Assistants write PySpark fast, and later they will save you hours. While you are learning, write the code first and ask afterwards. You cannot review code you could not have written, and the bugs that hurt look right: a union that matches columns by position, a `!=` that quietly skips nulls.

04 One dataset, messy on purpose

Every example uses the same two files, `employees.csv` and `departments.csv`, read into `emp` and `dept`. They carry what real data carries: 5 duplicate ids, 8 missing salaries and a salary of -5000. When a pattern trips over them, that is the lesson.

05 Practise from memory

Pick a section, close the sheet, and rebuild its patterns on your own data. Then check. Do it again next week. Reading a pattern feels like knowing it; writing it from memory is knowing it.

HOW TO READ A SECTION

Pattern and code

Each row pairs a one-line intent with code you can paste into a notebook cell.

Spark 3.4+

A badge gives the minimum Spark or Delta version a pattern needs.

Databricks · Fabric

Panels show what differs on each platform: paths, utilities, settings, limits.

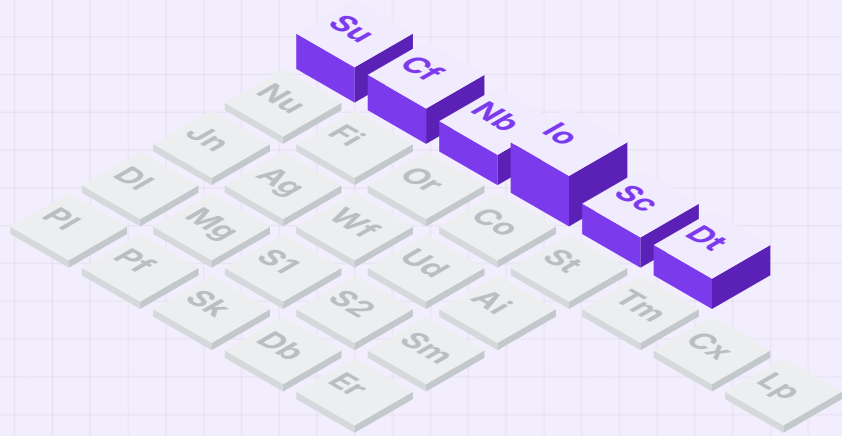
Watch out

What goes wrong in practice, and the fix, kept short.

Sanjay Chandra

Every open-source pattern was run on Spark 4.0.1 and 3.5.9; Databricks and Fabric patterns are marked.

CHAPTER 01 OF 05



Set up & Load.

Start a session, tune its settings, drive notebooks, then read data in with the right schema and types.

06

SECTIONS

42

PATTERNS

12

WATCH OUT

IN THIS CHAPTER

01	Setup & Import	04
02	Config Settings	06
03	Notebook Utilities	09
04	I/O	11
05	Schema	13
06	Data Types	15

SECTION 01 · SETUP & IMPORT

Setup & Import.

07
PATTERNS

01
WATCH OUT

SparkSession, imports, version, log level, stop.

PATTERN	CODE
01 Import SparkSession and types Always import types explicitly, avoids namespace collisions	<pre>from pyspark.sql import SparkSession from pyspark.sql.types import (StructType, StructField, StringType, IntegerType, DoubleType, DateType, TimestampType)</pre>
02 Import functions by name The everyday functions. Import others by name as you need them; alias sum, min and max so Python built-ins keep working	<pre>from pyspark.sql.functions import (col, lit, when, trim, lower, upper, concat_ws, to_date, current_date, datediff, count, countDistinct, avg, row_number, broadcast, sum as spark_sum, # keeps Python sum() usable min as spark_min, # keeps Python min() usable max as spark_max, # keeps Python max() usable) # Need another function? Add it to this list.</pre>
03 Import Window for ranking and running totals	<pre>from pyspark.sql.window import Window</pre>
04 Initialise Spark session <code>getOrCreate()</code> reuses an existing session if one is running	<pre>spark = SparkSession.builder \ .appName("MyApp") \ .config("spark.sql.adaptive.enabled", "true") \ .getOrCreate()</pre>
05 Check Spark version	<pre>print(spark.version)</pre>
06 Reduce log noise Set to WARN in dev, ERROR in production pipelines	<pre>spark.sparkContext.setLogLevel("WARN")</pre>
07 Stop the session Scripts and scheduled jobs only. In a Fabric or Databricks notebook the platform manages the session: do not call it there	<pre>spark.stop()</pre>

SECTION 01 · SETUP & IMPORT

CONTINUED

ON DATABRICKS AND FABRIC

DATABRICKS

Notebooks and jobs start with a ready `spark` session, so skip the builder and do not call `spark.stop()` in a notebook. Runtimes: 15.4 and 16.4 LTS run Spark 3.5, 17.3 LTS runs Spark 4.0, 18 LTS runs Spark 4.1. Photon is on by default for serverless, SQL warehouses and compute created in the UI, and falls back to standard Spark for UDFs and other unsupported operations.

MICROSOFT FABRIC

Notebooks start with a ready `spark` session. Runtime 2.0 (Spark 4.1) is GA and recommended, but new workspaces still default to Runtime 1.3 (Spark 3.5, end of support announced), so check your workspace setting: ANSI mode and `try_cast` depend on it. Turn on the native execution engine per notebook:

```
%%configure
{ "conf": { "spark.native.enabled": "true" } }
```

WATCH OUT**appName**

set a meaningful app name, it appears in Spark UI and makes debugging much easier

SECTION 02 · CONFIG SETTINGS

Config Settings.

07
PATTERNS

02
WATCH OUT

get, set, unset, isModifiable, the settings worth knowing, where to set them.

SETTINGS WORTH KNOWING (DEFAULTS CHECKED ON SPARK 3.5.9 AND 4.0.1)

SETTING	DEFAULT 3.5 / 4.X	CHANGE IT WHEN
<code>spark.sql.shuffle.partitions</code>	200	fewer for small data, more for very large shuffles
<code>spark.sql.adaptive.enabled</code>	true	leave on: AQE merges small partitions and splits skewed ones
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 MB	raise to broadcast a bigger small table; -1 turns it off
<code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code>	64 MB	AQE target size; ignored when merging small partitions while parallelismFirst is on (the default)
<code>spark.sql.files.maxPartitionBytes</code>	128 MB	lower for more parallel reads of large files
<code>spark.sql.ansi.enabled</code>	false / true	see Appendix B before changing it
<code>spark.sql.session.timeZone</code>	JVM local zone	pin to UTC so timestamps read the same everywhere
<code>spark.sql.sources.partitionOverwriteMode</code>	static	dynamic to replace only the partitions you write
<code>spark.sql.legacy.timeParserPolicy</code>	EXCEPTION / CORRECTED	LEGACY only for old date patterns that fail to parse
<code>spark.sql.execution.arrow.pyspark.enabled</code>	false	true to speed up toPandas() and pandas to Spark; default true from Spark 4.2

PATTERN	CODE
01 Read a setting Values always come back as strings	<pre>spark.conf.get("spark.sql.shuffle.partitions") spark.conf.get("spark.my.flag", "off") # if unset</pre>
02 Change a setting for this session A DataFrame already explained or run keeps its old plan: rebuild it after the change (see Spark Plans)	<pre>spark.conf.set("spark.sql.files.maxPartitionBytes", "64MB")</pre>

SECTION 02 · CONFIG SETTINGS

CONTINUED

PATTERN	CODE
03 Reset a setting to its default Back to 128 MB on Spark 3.5 and 4.x	<pre>spark.conf.unset("spark.sql.files.maxPartitionBytes")</pre>
04 Check before you set True for shuffle.partitions, False for static settings like warehouse.dir, which Spark 4 refuses with CANNOT_MODIFY_CONFIG	<pre>spark.conf.isModifiable("spark.sql.shuffle.partitions") spark.conf.isModifiable("spark.sql.warehouse.dir")</pre>
05 See every setting getAll: Spark 4.0+ SET -v adds a meaning and a since-version column	<pre>spark.sql("SET -v").show(truncate=False) settings = spark.conf.getAll # a dict</pre>
06 Enable adaptive query execution AQE optimises shuffle partitions and skew joins at runtime. On by default since Spark 3.2, so set it only on older runtimes	<pre>spark.conf.set("spark.sql.adaptive.enabled", "true") spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")</pre>
07 Pin the session time zone CSV timestamps without an offset are read in this zone: UTC and Asia/Kolkata put the same row 5.5 hours apart	<pre>spark.conf.set("spark.sql.session.timeZone", "UTC")</pre>

SECTION 02 · CONFIG SETTINGS

CONTINUED

ON DATABRICKS AND FABRIC

DATABRICKS

Set cluster-wide values in the compute's Spark config. Serverless accepts only six: shuffle.partitions (default auto), ansi.enabled, session.timeZone, legacy.timeParserPolicy, files.maxPartitionBytes and spark.databricks.execution.timeout.

MICROSOFT FABRIC

Set defaults in an Environment, or for one session with `%%configure` in the first cell; run later, it needs a session restart:

```
%%configure
{
  "conf": {"spark.sql.shuffle.partitions": "64"}
}
```

WATCH OUT**strings**

spark.conf.get returns text: compare with "true", not True

old configs

settings copied from older jobs can override better defaults in newer runtimes; Databricks recommends leaving most at their defaults

SECTION 03 · NOTEBOOK UTILITIES

Notebook Utilities.

06
PATTERNS

03
WATCH OUT

exit, run, runMultiple, parameters, secrets, files: dbutils vs notebookutils.

01 Exit with a return value Exit values are strings: send JSON and parse it in the caller

```
import json
```

DATABRICKS

```
dbutils.notebook.exit(json.dumps({"rows": emp.count(), "status": "ok"}))
```

MICROSOFT FABRIC

```
notebookutils.notebook.exit(json.dumps({"rows": emp.count(), "status": "ok"}))
```

02 Run a child notebook The number is a timeout in seconds; the result is whatever the child passed to exit

DATABRICKS

```
out = dbutils.notebook.run("load_employees", 600, {"run_date": "2026-09-27"})
```

MICROSOFT FABRIC

```
out = notebookutils.notebook.run("load_employees", 600, {"run_date":  
    "2026-09-27"})
```

03 Run notebooks in parallel

Fabric returns exitVal and exception per activity; on Databricks a thread pool is the usual pattern

```
depts = ["101", "102", "103"]
```

DATABRICKS

```
from concurrent.futures import ThreadPoolExecutor  
with ThreadPoolExecutor(max_workers=3) as pool:  
    outs = list(pool.map(  
        lambda d: dbutils.notebook.run("load_dept", 600, {"department_id": d}),  
        depts))
```

MICROSOFT FABRIC

```
dag = {"activities": [{"name": f"dept_{d}", "path": "load_dept",  
    "args": {"department_id": d}} for d in depts],  
    "concurrency": 3}  
outs = notebookutils.notebook.runMultiple(dag)
```

SECTION 03 · NOTEBOOK UTILITIES

CONTINUED

04 Read parameters

Job and `run()` arguments arrive through widgets on Databricks and the parameter cell on Fabric

DATABRICKS

```
dbutils.widgets.text("run_date", "2026-09-27")
run_date = dbutils.widgets.get("run_date")
```

MICROSOFT FABRIC

```
# in a cell marked "parameters": defaults, replaced by run() or a pipeline
run_date = "2026-09-27"
```

05 Read a secret Databricks redacts secret values in notebook output

DATABRICKS

```
pwd = dbutils.secrets.get(scope="hr", key="db-password")
```

MICROSOFT FABRIC

```
pwd = notebookutils.credentials.getSecret("https://<vault>.vault.azure.net/",
    "db-password")
```

06 Work with files rm with True deletes a folder and everything in it

DATABRICKS

```
dbutils.fs.ls("/Volumes/main/hr/raw/")
dbutils.fs.cp("/Volumes/main/hr/raw/employees.csv", "/Volumes/main/hr/archive/
    employees.csv")
dbutils.fs.rm("/Volumes/main/hr/tmp/", True)
```

MICROSOFT FABRIC

```
notebookutils.fs.ls("Files/raw")
notebookutils.fs.cp("Files/raw/employees.csv", "Files/archive/employees.csv")
notebookutils.fs.rm("Files/tmp", True)
```

WATCH OUT

mssparkutils

renamed notebookutils in Fabric: old code still runs, but the namespace will be retired. Azure Synapse still uses mssparkutils

exit()

on Fabric, call it in its own cell, not inside try/except; on Databricks it does not end the run while a stream is running

driver only

these run on the driver; Databricks warns that calling dbutils on executors can fail, so keep them out of UDFs and foreachPartition

SECTION 04 · I/O

I/O.

10
PATTERNS

03
WATCH OUT

Parquet, CSV, JSON, partitioning, managed tables.

PATTERN	CODE
01 Read Parquet Best format for Spark, columnar, compressed and schema-aware	<pre>emp = spark.read.parquet("path/to/employees")</pre>
02 Write Parquet	<pre>emp.write.mode("overwrite").parquet("path/to/employees")</pre>
03 Read CSV inferSchema is slow on large files, prefer an explicit schema in production	<pre>emp = spark.read \ .option("header", "true") \ .option("inferSchema", "true") \ .csv("employees.csv")</pre>
04 Read JSON multiline required for standard nested JSON objects spanning multiple lines	<pre>emp = spark.read.option("multiline", "true") .json("employees.json")</pre>
05 Write partitioned data Partition by columns used in WHERE filters for maximum query speed	<pre>emp.write.partitionBy("department_id").parquet("path/to/employees_by_dept")</pre>
06 Save as managed table Registers the table in the metastore (Unity Catalog on Databricks, the lakehouse on Fabric), queryable with <code>spark.sql()</code>	<pre>emp.write.saveAsTable("hr.employees")</pre>
07 Overwrite only the partitions you write Static mode, the default, deletes the other partitions: on your data 1 of 9 was left. Dynamic keeps all 9	<pre>spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic") emp.filter(col("department_id") == 101) \ .write.mode("overwrite") \ .partitionBy("department_id") \ .parquet("path/to/employees_by_dept")</pre>

SECTION 04 · I/O

CONTINUED

PATTERN

CODE

08 Read Parquet files with different columns

Without mergeSchema, a column that exists in only some files can be missing from the result

```
emp = spark.read.option("mergeSchema", "true") \
    .parquet("path/to/employees")
```

09 Read a table by name

Use the catalog name, not a path: three parts on Unity Catalog (main.hr.employees)

```
emp = spark.read.table("hr.employees")
high = spark.sql("SELECT * FROM hr.employees WHERE salary > 100000")
```

10 Write CSV with a header

Writes one file per partition into the folder; for one small file, coalesce(1) first

```
emp.write.mode("overwrite").option("header", True)
    .csv("path/to/employees_csv")
```

ON DATABRICKS AND FABRIC

DATABRICKS

Files live in Unity Catalog volumes; tables use a three-part name.

```
emp = spark.read.csv("/Volumes/main/hr/raw/employees.csv",
                    header=True, inferSchema=True)
emp.write.saveAsTable("main.hr.employees") # catalog.schema.table
```

MICROSOFT FABRIC

Relative paths point at the default lakehouse: Files/ for files, saveAsTable for Delta tables. Other lakehouses need an ABFS path.

```
emp = spark.read.csv("Files/employees.csv", header=True, inferSchema=True)
emp.write.format("delta").saveAsTable("employees")
# abfss://<workspace>@onelake.dfs.fabric.microsoft.com/
# <lakehouse>.Lakehouse/Files/employees.csv
```

WATCH OUT

Parquet

default to Parquet or Delta for all persistent storage, never CSV in production

inferSchema

always costs a full scan of the file, pass an explicit schema for large CSVs

overwrite

mode overwrite replaces the entire dataset, use append to add incrementally

SECTION 05 · SCHEMA

Schema.

06
PATTERNS

01
WATCH OUT

StructType, apply a schema, printSchema, nullable, metadata.

PATTERN

CODE

01 Define a schema explicitly

Always prefer explicit schema over inferSchema in production

```
schema = StructType([
    StructField("employee_id", IntegerType(), False),
    StructField("employee_name", StringType(), True),
    StructField("job_title", StringType(), True),
    StructField("department_id", IntegerType(), True),
    StructField("hire_date", DateType(), True),
    StructField("salary", IntegerType(), True),
    StructField("updated_at", TimestampType(), True),
])
```

02 Apply schema when reading

Fastest read method, skips the inferSchema scan entirely

```
emp = spark.read.schema(schema).option("header", "true")
    .csv("employees.csv")
```

03 Print schema and column info

```
emp.printSchema()           # tree format
print(emp.dtypes)           # list of (name, type) tuples
print(emp.columns)          # list of column names
```

04 Force all fields to nullable

Useful when merging schemas from different sources

```
new_schema = StructType([
    StructField(f.name, f.dataType, True)
    for f in emp.schema.fields
])
```

05 Add metadata to a field

Metadata is stored in the schema and survives write/read cycles

```
emp = emp.withColumn("salary",
    col("salary").alias("salary", metadata={"description":
    "annual salary"}))
)
print(emp.schema["salary"].metadata)  # {'description':
    'annual salary'}
```

06 Build a small DataFrame from Python data

A DDL string sets the column names and types in one go; handy for lookups and tests

```
bands = spark.createDataFrame(
    [(0, 80000, "A"), (80000, 120000, "B"), (120000,
    200000, "C")],
    "low int, high int, band string")
```

SECTION 05 · SCHEMA

CONTINUED

ON DATABRICKS AND FABRIC

DATABRICKS

ANSI mode is the Spark 4 default, so an invalid cast raises an error on Spark 4 runtimes (17.3 LTS and later). Check before relying on either behaviour:

```
spark.conf.get("spark.sql.ansi.enabled")
```

MICROSOFT FABRIC

Runtime 1.3 is Spark 3.5 (ANSI off by default, try_cast not available on Column). Runtime 2.0 is Spark 4.1. Check the setting the same way:

```
spark.conf.get("spark.sql.ansi.enabled")
```

WATCH OUT**StructField**

third argument is nullable, set False for primary keys and required fields

SECTION 06 · DATA TYPES

Data Types.

06
PATTERNS

02
WATCH OUT

Type names, cast, decimal, display text, dates, JSON, many columns.

TYPE NAMES: USE THE CLASS, OR THE DDL NAME AS A STRING IN CAST()

SPARK TYPE	DDL NAME	PYTHON VALUE	IN YOUR DATA
<code>IntegerType</code>	<code>int</code>	<code>int</code>	<code>employee_id</code> , <code>salary</code>
<code>LongType</code>	<code>bigint</code>	<code>int</code>	
<code>DoubleType</code>	<code>double</code>	<code>float</code>	
<code>DecimalType(12, 2)</code>	<code>decimal(12,2)</code>	<code>Decimal</code>	
<code>StringType</code>	<code>string</code>	<code>str</code>	<code>employee_name</code> , <code>job_title</code>
<code>BooleanType</code>	<code>boolean</code>	<code>bool</code>	
<code>DateType</code>	<code>date</code>	<code>datetime.date</code>	<code>hire_date</code>
<code>TimestampType</code>	<code>timestamp</code>	<code>datetime.datetime</code>	<code>updated_at</code>
<code>ArrayType(StringType())</code>	<code>array<string></code>	<code>list</code>	
<code>MapType(StringType(), StringType())</code>	<code>map<string,string></code>	<code>dict</code>	
<code>StructType([...])</code>	<code>struct<a:int,b:string></code>	<code>Row</code>	

PATTERN	CODE
01 Cast a column to a new type try_cast: Spark 4.0+ With ANSI mode on (the default from Spark 4.0) an invalid value raises an error; <code>try_cast</code> returns null instead	<pre>emp = emp.withColumn("department_id", col("department_id") .cast(IntegerType())) emp = emp.withColumn("salary", col("salary") .cast("double")) emp = emp.withColumn("salary", col("salary") .try_cast("int")) # null, not an error</pre>
02 Keep money exact with decimal Division returns a double: 93,844 / 12 is 7820.333333333333, as <code>decimal(12,2)</code> it is 7820.33	<pre>emp = emp.withColumn("monthly_salary", (col("salary") / 12).cast("decimal(12,2)"))</pre>

SECTION 06 · DATA TYPES

CONTINUED

PATTERN

CODE

03 Number to display text

format_number adds
thousands separators:
98904 becomes "98,904"

```
emp = emp.withColumn("salary_text",
    format_number(col("salary"), 0))
emp = emp.withColumn("salary_str", col("salary")
    .cast("string"))
```

04 Timestamp to date and back

A date cast to a timestamp
lands at midnight

```
emp = emp.withColumn("updated_day", col("updated_at")
    .cast("date"))
emp = emp.withColumn("hired_ts", col("hire_date")
    .cast("timestamp"))
```

05 Struct to JSON text and back

from_json takes the schema
as a DDL string

```
emp = emp.withColumn("json", to_json(struct("employee_id",
    "salary")))
emp = emp.withColumn("parsed", from_json(col("json"),
    "employee_id INT, salary INT"))
```

06 Change the type of many columns

One select for all of them;
here every *_id column
becomes a string

```
emp = emp.select([
    col(c).cast("string") if c.endswith("_id") else col(c)
    for c in emp.columns
])
```

WATCH OUT

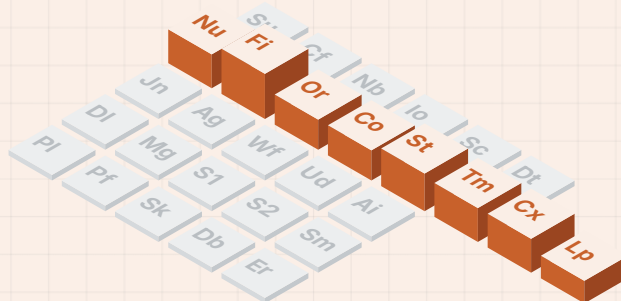
cast()

invalid values raise an error under ANSI mode (Spark 4.0 default) and become null when ANSI is off; use try_cast when bad values are expected

boolean casts

"true", "Y", "yes" and "1" cast to true; other text is null on Spark 3.5 and an error on Spark 4

CHAPTER 02 OF 05



Clean & Shape.

Fix nulls and duplicates, filter, sort, reshape columns, strings, dates and nested data, and loop the right way.

08

SECTIONS

71

PATTERNS

13

WATCH OUT

IN THIS CHAPTER

07	Nulls & Duplicates	18
08	Filtering	19
09	Ordering	21
10	Column Operations	22
11	String Operations	23
12	Date & Time	25
13	Complex Data Types	26
14	Loops	28

SECTION 07 · NULLS & DUPLICATES

Nulls & Duplicates.

09
PATTERNS

01
WATCH OUT

dropDuplicates, na.drop, na.fill, null counts, trim.

PATTERN	CODE
01 Drop all duplicate rows Checks every column, equivalent to SELECT DISTINCT	<pre>emp = emp.dropDuplicates()</pre>
02 Drop duplicates on specific columns Which duplicate survives is not guaranteed; to keep the latest, use row_number over a window (Window Functions 09)	<pre>emp = emp.dropDuplicates(["employee_id"]) emp = emp.dropDuplicates(["employee_id", "updated_at"])</pre>
03 Drop rows with any null	<pre>emp = emp.na.drop(how="any")</pre>
04 Drop rows where all columns are null	<pre>emp = emp.na.drop(how="all")</pre>
05 Drop nulls in specific columns	<pre>emp = emp.na.drop(subset=["salary", "hire_date"])</pre>
06 Fill nulls with a default value	<pre>emp = emp.na.fill(0) emp = emp.na.fill({"salary": 0, "job_title": "Unknown"})</pre>
07 Replace specific values Useful for replacing sentinel values like empty strings or -1	<pre>emp = emp.replace("", None) emp = emp.replace(-5000, None, subset=["salary"])</pre>
08 Count nulls per column Run this early as a data quality check	<pre>null_counts = emp.select([count(when(col(c).isNull(), c)).alias(c) for c in emp.columns]) null_counts.show()</pre>

SECTION 07 · NULLS & DUPLICATES

CONTINUED

PATTERN

CODE

09 Trim whitespace and standardise case

Do this before deduplication to catch near-duplicates

```
emp = emp.withColumn("employee_name",
    trim(col("employee_name")))
emp = emp.withColumn("job_title",
    lower(col("job_title")))
```

WATCH OUT

`dropDuplicates`

specify columns explicitly, dropping on all columns may keep unwanted duplicates

SECTION 08 · FILTERING

Filtering.

12

PATTERNS

01

WATCH OUT

Equality, AND, OR, isin, null checks, regex, date range.

PATTERN

CODE

01 Simple equality filter

`.where()` is an exact alias for `.filter()`, both compile identically

```
eng = emp.filter(col("department_id") == 101)
eng = emp.where(col("department_id") == 101) # identical
```

02 Multiple conditions, AND

Parentheses are mandatory around each condition in PySpark

```
result = emp.filter(
    (col("salary") > 100000) &
    (col("department_id") == 101)
)
```

03 Multiple conditions, OR

```
result = emp.filter(
    (col("department_id") == 103) |
    (col("department_id") == 105)
)
```

04 Filter using a list, IN

`isin()` accepts *args or an unpacked list

```
result = emp.filter(col("department_id").isin(101, 102, 107))
```

05 Filter using a list, NOT IN

Prefix ~ to negate `isin()`, equivalent to SQL NOT IN

```
result = emp.filter(~col("department_id").isin(101, 102, 107))
```

SECTION 08 · FILTERING

CONTINUED

PATTERN	CODE
06 Null check, is null	<pre>no_salary = emp.filter(col("salary").isNull())</pre>
07 Null check, is not null	<pre>result = emp.filter(col("salary").isNotNull())</pre>
08 String pattern matching <code>contains()</code> / <code>startswith()</code> / <code>endswith()</code> / <code>rlike()</code> for regex	<pre>result = emp.filter(col("job_title").contains("Engineer")) result = emp.filter(col("employee_name") .startswith("Dr. ")) result = emp.filter(col("employee_name") .rlike(r"^(Mr Mrs Ms Dr)\. ")) # regex</pre>
09 Negation with tilde ~ inverts any boolean column expression	<pre>result = emp.filter(~(col("department_id") == 101))</pre>
10 Filter on a date column Cast string columns with <code>to_date()</code> before comparing dates	<pre>result = emp.filter(col("hire_date") >= lit("2023-01-01")) result = emp.filter(to_date(col("updated_at")) >= lit("2025-01-01"))</pre>
11 Filter between a range Inclusive on both ends, equivalent to <code>col >= low & col <= high</code>	<pre>result = emp.filter(col("salary").between(80000, 120000))</pre>
12 Filter in SQL with parameters Spark 3.4+ Values are bound, not pasted into the SQL text, so input cannot change the query	<pre>result = spark.sql("SELECT * FROM {emp} " "WHERE department_id = :dept AND salary > :min_salary", args={"dept": 101, "min_salary": 120000}, emp=emp)</pre>

WATCH OUT

`col()`
use `col()` for every column reference; it avoids ambiguity once DataFrames are joined

SECTION 09 · ORDERING

Ordering.

08
PATTERNS

02
WATCH OUT

orderBy, sort, nulls first/last, sortWithinPartitions.

PATTERN	CODE
01 Sort ascending (default) <code>orderBy()</code> and <code>sort()</code> are exact aliases	<pre>result = emp.orderBy("salary") result = emp.sort("salary") # identical</pre>
02 Sort descending	<pre>result = emp.orderBy(col("salary").desc())</pre>
03 Sort by multiple columns Mix asc and desc across columns freely	<pre>result = emp.orderBy(col("department_id").asc(), col("salary").desc())</pre>
04 Sort with nulls first Useful for surfacing missing data at the top	<pre>result = emp.orderBy(col("salary").desc_nulls_first())</pre>
05 Sort with nulls last Ascending sorts put nulls first by default; this moves them to the end	<pre>result = emp.orderBy(col("salary").asc_nulls_last())</pre>
06 Sort within partitions only No global shuffle, much faster. Use before writing files	<pre>result = emp.sortWithinPartitions("hire_date")</pre>
07 Limit top N rows Always sort before limiting to get deterministic results	<pre>top_5 = emp.orderBy(col("salary").desc()).limit(5)</pre>
08 Get first or last N rows <code>first()</code> returns a Row object, not a DataFrame	<pre>first_row = emp.first() # single Row object rows = emp.take(5) # list of Row objects emp_head = emp.limit(5) # DataFrame</pre>

WATCH OUT

nulls placement

ascending puts nulls first, descending puts them last; say `nulls_first` or `nulls_last` explicitly

limit()

always pair with `orderBy`, without it `limit` returns arbitrary rows

SECTION 10 · COLUMN OPERATIONS

Column Operations.

08
PATTERNS

02
WATCH OUT

withColumn, lit, rename, CASE WHEN, drop, select.

PATTERN	CODE
01 Add a constant column <code>lit()</code> wraps a scalar value into a column expression	<pre>emp = emp.withColumn("report_date", lit("2025-04-30")) emp = emp.withColumn("is_active", lit(True))</pre>
02 Add a calculated column	<pre>emp = emp.withColumn("monthly_salary", col("salary") / 12)</pre>
03 Rename a column	<pre>emp = emp.withColumnRenamed("employee_name", "name")</pre>
04 Rename multiple columns Spark 3.4+ <code>withColumnsRenamed</code> renames several columns in one call, instead of chaining <code>withColumnRenamed()</code>	<pre>rename_map = {"employee_name": "name", "job_title": "title"} emp = emp.withColumnsRenamed(rename_map)</pre>
05 Conditional logic, CASE WHEN Chain as many <code>.when()</code> conditions as needed	<pre>emp = emp.withColumn("salary_band", when(col("salary") >= 120000, "High") .when(col("salary") >= 80000, "Mid") .otherwise("Low"))</pre>
06 Drop one or more columns	<pre>emp = emp.drop("updated_at") emp = emp.drop("job_title", "hire_date", "updated_at")</pre>
07 Select and reorder columns <code>select()</code> can also rename via <code>.alias()</code>	<pre>emp = emp.select("employee_id", "employee_name", "salary") emp = emp.select(col("employee_id"), col("salary") .alias("annual_salary"))</pre>

SECTION 10 · COLUMN OPERATIONS

CONTINUED

PATTERN

CODE

08 Round, and take the first non-null

`coalesce()` here is the column function, not `DataFrame.coalesce()`; alias round so Python `round()` keeps working

```
from pyspark.sql.functions import coalesce, round as spark_round
emp = emp.withColumn("salary_k", spark_round(col("salary") / 1000, 1))
emp = emp.withColumn("salary_or_0", coalesce(col("salary"), lit(0)))
```

WATCH OUT

`withColumn()`

each call adds a projection to the plan; to add many columns use `withColumns({...})` or a single `select()`

`.otherwise()`

always add `.otherwise()` to `when()` chains, omitting it returns null for unmatched rows

SECTION 11 · STRING OPERATIONS

String Operations.

10

PATTERNS

01

WATCH OUT

Case, trim, substring, split, regex replace and extract.

PATTERN

CODE

01 Change case

```
emp = emp.withColumn("name_lower", lower(col("employee_name")))
emp = emp.withColumn("name_upper", upper(col("employee_name")))
emp = emp.withColumn("name_title", initcap(col("employee_name")))
```

02 Trim whitespace

Always trim before joins or comparisons

```
emp = emp.withColumn("employee_name", trim(col("employee_name")))
emp = emp.withColumn("name_ltrim", ltrim(col("employee_name")))
emp = emp.withColumn("name_rtrim", rtrim(col("employee_name")))
```

03 Concatenate strings

`concat_ws` skips nulls, `concat()` returns null if any arg is null

```
emp = emp.withColumn("name_and_title", concat_ws(" ", col("employee_name"), col("job_title")))
)
```

SECTION 11 · STRING OPERATIONS

CONTINUED

PATTERN

CODE

04 Substring extraction

Index starts at 1 in PySpark,
not 0

```
emp = emp.withColumn("title_prefix",
    substring(col("job_title"), 1, 3))
emp = emp.withColumn("name_start", col("employee_name")
    .substr(1, 4))
```

05 Split string into array

```
emp = emp.withColumn("name_parts",
    split(col("employee_name"), " "))
```

06 Regex replace

Strip a leading title such as
Mr. or Dr. from the name

```
emp = emp.withColumn("clean_name",
    regexp_replace(col("employee_name"),
    r"^(Mr|Mrs|Ms|Dr)\.s+", ""))
```

07 Regex extract

Capture group 1 is
extracted, returns empty
string if no match

```
emp = emp.withColumn("name_prefix",
    regexp_extract(col("employee_name"),
    r"^(Mr|Mrs|Ms|Dr)\.", 1))
```

08 Check string length

```
emp = emp.withColumn("name_len",
    length(col("employee_name")))
```

09 Contains, starts, ends checks

```
emp = emp.withColumn("is_engineer", col("job_title")
    .contains("Engineer"))
emp = emp.withColumn("is_dr", col("employee_name")
    .startswith("Dr.))
emp = emp.withColumn("is_manager", col("job_title")
    .endsWith("Manager"))
```

10 Left-pad to fixed width

Useful for zero-padding IDs
or codes

```
emp = emp.withColumn("id_padded", lpad(col("employee_id")
    .cast("string"), 8, "0"))
```

WATCH OUT

regexp_extract

returns empty string on no match, not null, filter on length > 0 to detect misses

SECTION 12 · DATE & TIME

Date & Time.

09

PATTERNS

02

WATCH OUT

to_date, date_format, datediff, trunc, unix timestamp.

PATTERN

CODE

01 Get current date and timestamp

```
emp = emp.withColumn("today", current_date())
emp = emp.withColumn("now", current_timestamp())
```

02 Convert string to date

Format must match the string exactly

```
# emp_raw: employees.csv read without inferSchema, every
column is a string
emp = emp_raw.withColumn("hire_date",
    to_date(col("hire_date"), "yyyy-MM-dd"))
emp = emp.withColumn("updated_at",
    to_timestamp(col("updated_at"), "yyyy-MM-dd HH:mm:ss"))
```

03 Format date as string

```
emp = emp.withColumn("updated_str",
    date_format(col("updated_at"), "MM/dd/yyyy"))
emp = emp.withColumn("hire_month",
    date_format(col("hire_date"), "MMM yyyy"))
```

04 Date arithmetic, add and subtract

date_add and date_sub work in whole days only

```
emp = emp.withColumn("probation_end",
    date_add(col("hire_date"), 90))
emp = emp.withColumn("offer_date",
    date_sub(col("hire_date"), 30))
emp = emp.withColumn("first_review",
    add_months(col("hire_date"), 12))
```

05 Date difference in days

Returns end_date minus start_date as an integer

```
emp = emp.withColumn("days_employed",
    datediff(current_date(), col("hire_date"))
)
```

06 Extract date components

```
emp = emp.withColumn("hire_year", year(col("hire_date")))
emp = emp.withColumn("hire_month",
    month(col("hire_date")))
emp = emp.withColumn("hire_day",
    dayofmonth(col("hire_date")))
emp = emp.withColumn("hire_dow",
    dayofweek(col("hire_date"))) # 1=Sun
```

SECTION 12 · DATE & TIME

CONTINUED

PATTERN	CODE
07 Truncate to start of period Useful for monthly and weekly reporting aggregations	<pre>emp = emp.withColumn("hire_month_start", trunc(col("hire_date"), "month")) emp = emp.withColumn("hire_year_start", trunc(col("hire_date"), "year")) emp = emp.withColumn("update_week", date_trunc("week", col("updated_at")))</pre>
08 Unix timestamp conversion unix_timestamp returns seconds since epoch as a long	<pre>emp = emp.withColumn("updated_unix", unix_timestamp(col("updated_at"))) emp = emp.withColumn("updated_day", from_unixtime(col("updated_unix"), "yyyy-MM-dd"))</pre>
09 Filter on date range Wrap date literals in <code>lit()</code> for reliable comparisons	<pre>hired_2024 = emp.filter((col("hire_date") >= lit("2024-01-01")) & (col("hire_date") < lit("2025-01-01")))</pre>

WATCH OUT

`to_date()`
always pass the format; with ANSI mode on (Spark 4.0 default) a string that does not match raises an error

`datediff()`
argument order is (end, start), swap them and you get a negative number

SECTION 13 · COMPLEX DATA TYPES

Complex Data Types.

09
PATTERNS

02
WATCH OUT

explode, array, map, struct, element_at.

PATTERN	CODE
01 Explode array into rows Creates one row per element, row count increases	<pre>emp = emp.withColumn("title_words", split(col("job_title"), " ")) words = emp.withColumn("word", explode(col("title_words")))</pre>
02 Explode, keep null arrays explode drops nulls, explode_outer keeps them as a null row	<pre>words = emp.withColumn("word", explode_outer(col("title_words")))</pre>

SECTION 13 · COMPLEX DATA TYPES

CONTINUED

PATTERN	CODE
03 Check array contains value	<pre>seniors = emp.filter(array_contains(col("title_words"), "Senior"))</pre>
04 Get array size	<pre>emp = emp.withColumn("n_words", size(col("title_words")))</pre>
05 Get element at index Index is 1-based, negative index counts from the end	<pre>emp = emp.withColumn("first_word", element_at(col("title_words"), 1)) emp = emp.withColumn("last_word", element_at(col("title_words"), -1))</pre>
06 Create a map from columns	<pre>dept = dept.withColumn("props", create_map(lit("location"), col("location"), lit("region"), col("region")))</pre>
07 Get map keys and values	<pre>dept = dept.withColumn("prop_keys", map_keys(col("props"))) dept = dept.withColumn("prop_values", map_values(col("props")))</pre>
08 Access nested struct field Use dot notation in <code>col()</code> for nested access	<pre># office is a struct<location, region>, built in pattern 09 dept = dept.withColumn("city", col("office.location")) dept = dept.withColumn("region", col("office.region"))</pre>
09 Collapse columns into a struct	<pre>dept = dept.withColumn("office", struct(col("location"), col("region")))</pre>

WATCH OUT

`explode()`

increases row count, always be aware of the fan-out when exploding large arrays

`struct()`

useful for grouping related columns before writing, struct fields survive parquet round-trips

SECTION 14 · LOOPS

Loops.

06
PATTERNS

02
WATCH OUT

Many columns, many DataFrames, many tables, rows on the driver or executors.

PATTERN

CODE

01 Same change to many columns Spark 3.3+

One projection for all columns; a withColumn loop adds one per call

```
str_cols = [c for c, t in emp.dtypes if t == "string"]
emp = emp.withColumns({c: trim(col(c)) for c in str_cols})
```

02 Union a list of DataFrames

137 rows for departments
101 to 103 on your data

```
from functools import reduce
parts = [emp.filter(col("department_id") == d)
         for d in (101, 102, 103)]
combined = reduce(lambda a, b: a.unionByName(b), parts)
```

03 Build a filter from a list

167 titles contain Engineer,
Architect or Analyst

```
from functools import reduce
keywords = ["Engineer", "Architect", "Analyst"]
cond = reduce(lambda a, b: a | b, [col("job_title")
                                   .contains(k) for k in keywords])
tech = emp.filter(cond)
```

04 Loop over tables on the driver

Each pass is its own Spark
job: fine for tens of tables

```
for name, df in {"employees": emp,
                 "departments": dept}.items():
    df.write.mode("overwrite").parquet(f"path/to/{name}")
```

05 Iterate rows on the driver

Streams one partition at a
time, so the driver never
holds the whole table

```
for row in emp.select("employee_id", "salary")
               .toLocalIterator():
    if row.salary is None:
        print("missing salary:", row.employee_id)
```

06 Side effects on the executors

Runs on the executors and
returns nothing: use it to
push rows to another
system

```
def send(rows):
    for r in rows:
        pass # e.g. post r.asDict() to an API
emp.foreachPartition(send)
```

WATCH OUT

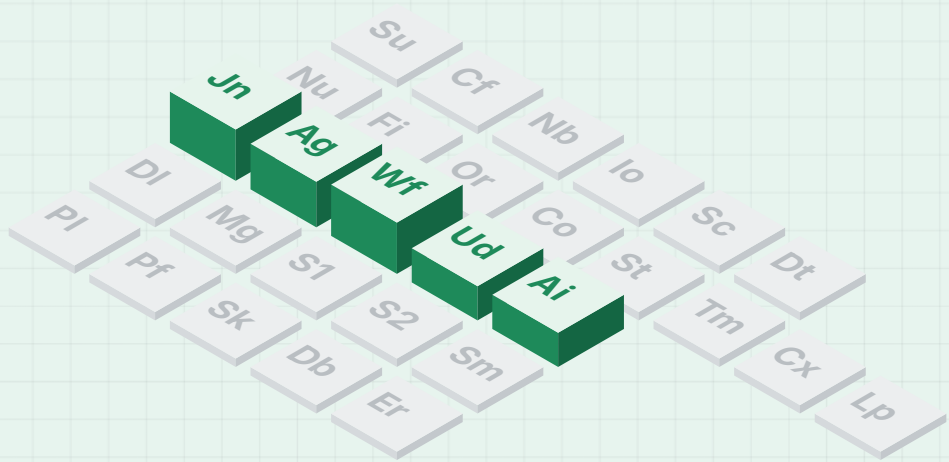
collect()

pulls every row onto the driver at once: fine for small
results, an out-of-memory error on big ones

row loops

a Python for over rows runs one row at a time on one
machine; write it as a column expression or a join
instead

CHAPTER 03 OF 05



Combine & Enrich.

Join, aggregate and window, then go further with UDFs and the AI functions built into Databricks and Fabric.

05

SECTIONS

38

PATTERNS

08

WATCH OUT

IN THIS CHAPTER

15	Joins	30
16	Aggregations	36
17	Window Functions	38
18	UDFs	45
19	AI Functions	46

SECTION 15 · JOINS

Joins.

09
PATTERNS

01
WATCH OUT

Inner, left, outer, semi, anti, broadcast, ambiguity fix.

THE DATA: YOUR EMPLOYEES.CSV AND DEPARTMENTS.CSV

```
emp = spark.read.csv("employees.csv", header=True, inferSchema=True)
dept = spark.read.csv("departments.csv", header=True, inferSchema=True)
```

DEPARTMENTS.CSV · ALL 10 ROWS

department_id	department_name	location	region
101	Engineering	London	EMEA
102	Data & Analytics	Singapore	APAC
103	Sales	New York	AMER
104	Human Resources	London	EMEA
105	Finance	New York	AMER
106	Product Management	Singapore	APAC
107	Cloud Infrastructure	Sydney	APAC
108	Cybersecurity	New York	AMER
109	Legal & Compliance	London	EMEA
110	Executive Office	New York	AMER

- 332 employee rows, 327 unique employee_id values (5 employees appear twice)
- 10 departments. Executive Office (110) has no employees
- 8 null salaries, 1 negative, 1 of 9,999,999
- 10 null hire dates, 5 null updated_at values

EMPLOYEES.CSV · FIRST 6 OF 332 ROWS

employee_id	employee_name	job_title	department_id	hire_date	salary
1001	Danielle Johnson	Junior Engineer	101	2018-04-13	98,904
1139	Daniel Tran	HR Manager	104	2023-11-19	76,583
1096	Robert Ramirez	Data Analyst	102	2023-03-19	92,517
1218	Ms. April Williams	Associate PM	106	2020-06-08	-5000
1236	Rick Daniels	Principal Architect	107	2022-08-22	119,423
1255	Brent Gray	Cloud Architect	107	2024-09-20	101,975

SECTION 15 · JOINS

CONTINUED

A SMALL SLICE SO EVERY RESULT FITS ON THE PAGE

Five employees and three departments, chosen so that Sales (103) and Finance (105) have no matching department row, and Executive Office (110) has no matching employee.

```
emp_s = emp.filter(col("employee_id").isin(1001, 1022, 1096, 1133, 1173)) \
    .select("employee_id", "employee_name", "department_id")
dept_s = dept.filter(col("department_id").isin(101, 102, 110)) \
    .select("department_id", "department_name")
```

EMP_S

employee_id	employee_name	department_id
1001	Danielle Johnson	101
1022	Jeremy Roberts	101
1096	Robert Ramirez	102
1133	Martha Warner	103
1173	Mark Lin	105

DEPT_S

department_id	department_name
101	Engineering
102	Data & Analytics
110	Executive Office

Only in emp_s (left)
 Only in dept_s (right)
 null Filled in: no match
 department_id Join key

SIX JOIN TYPES ON THE SLICE, WITH THE ROW COUNT ON THE FULL FILES

Inner

3 ROWS

Only employees whose department exists in dept_s.

```
emp_s.join(dept_s,
    "department_id", "inner")
```

Full files: 332 rows

department_id	employee_id	employee_name	department_name
101	1001	Danielle Johnson	Engineering
101	1022	Jeremy Roberts	Engineering
102	1096	Robert Ramirez	Data & Analytics

Left outer

5 ROWS

Every employee. department_name is null where there is no match.

```
emp_s.join(dept_s,
    "department_id", "left")
```

Full files: 332 rows

department_id	employee_id	employee_name	department_name
101	1001	Danielle Johnson	Engineering
101	1022	Jeremy Roberts	Engineering
102	1096	Robert Ramirez	Data & Analytics
103	1133	Martha Warner	null
105	1173	Mark Lin	null

Right outer

4 ROWS

Every department. Employee columns are null for Executive Office.

```
emp_s.join(dept_s,
    "department_id", "right")
```

Full files: 333 rows

department_id	employee_id	employee_name	department_name
101	1001	Danielle Johnson	Engineering
101	1022	Jeremy Roberts	Engineering
102	1096	Robert Ramirez	Data & Analytics
110	null	null	Executive Office

SECTION 15 · JOINS

CONTINUED

Full outer

6 ROWS

Everything from both sides, matched where possible.

```
emp_s.join(dept_s,
           "department_id", "full")
```

Full files: 333 rows

department_id	employee_id	employee_name	department_name
101	1001	Danielle Johnson	Engineering
101	1022	Jeremy Roberts	Engineering
102	1096	Robert Ramirez	Data & Analytics
103	1133	Martha Warner	null
105	1173	Mark Lin	null
110	null	null	Executive Office

Left semi

3 ROWS

Employees that have a match. emp_s columns only, never duplicated.

```
emp_s.join(dept_s,
           "department_id",
           "left_semi")
```

Full files: 332 rows

department_id	employee_id	employee_name
101	1001	Danielle Johnson
101	1022	Jeremy Roberts
102	1096	Robert Ramirez

Left anti

2 ROWS

Employees with no match. emp_s columns only.

```
emp_s.join(dept_s,
           "department_id",
           "left_anti")
```

Full files: 0 rows

department_id	employee_id	employee_name
103	1133	Martha Warner
105	1173	Mark Lin

DECISION MATRIX

how=	Unmatched left	Unmatched right	Columns	Slice	Full files	SQL	Also accepted
"inner"	Dropped	Dropped	Both	3	332	INNER JOIN	.
"left"	Kept, right side null	Dropped	Both	5	332	LEFT OUTER JOIN	leftouter, left_outer
"right"	Dropped	Kept, left side null	Both	4	333	RIGHT OUTER JOIN	rightouter, right_outer
"full"	Kept	Kept	Both	6	333	FULL OUTER JOIN	outer, fullouter, full_outer
"left_semi"	Dropped	Not returned	Left only	3	332	WHERE EXISTS (...)	semi, leftsemi
"left_anti"	Only these are kept	Not returned	Left only	2	0	WHERE NOT EXISTS (...)	anti, leftanti
"cross"	Every pairing	Every pairing	Both	15	3,320	CROSS JOIN	crossJoin(dept_s)

Reading the full-file column. Every employee has a valid department, so inner, left and left_semi all return 332 and left_anti returns 0. Right and full return 333: the extra row is Executive Office, which has nobody. Cross join is $332 \times 10 = 3,320$. Every alias listed was run and returns the same rows as its canonical name.

SECTION 15 · JOINS

CONTINUED

FIVE BEHAVIOURS WORTH KNOWING, EACH RUN ON YOUR DATA

A Null keys never match

Your files have no null department_id, but new data often will. In a join condition null == null is not true, so those rows drop out. Use eqNullSafe when nulls should pair up.

A	B
department_id a	department_id b
101 a1	101 b1
null a2	null b2

```
a.join(b, a.department_id == b
      .department_id)
# 1 row
a.join(b, a.department_id
      .eqNullSafe(b.department_id))
# 2 rows
```

= =	EQNULLSAFE
department_id a b	department_id a b
101 a1 b1	101 a1 b1
	null a2 b2

B Join by name or by expression

A column name keeps one key column. An expression keeps both, so drop or rename one afterwards.

```
emp_s.join(dept_s, "department_id")
# department_id, employee_id,
# employee_name, department_name

emp_s.join(dept_s,
           emp_s.department_id == dept_s
           .department_id)
# employee_id, employee_name,
# department_id, department_id,
# department_name
```

C Duplicate keys multiply rows

employee_id 1014, 1066, 1139, 1154, 1172 each appear twice. Joined to itself, each becomes $2 \times 2 = 4$ rows: 332 rows in, 342 out. Deduplicate first (Window Functions, pattern 09).

EMPLOYEE 1172 IN EMPLOYEES.CSV

employee_id	employee_name	salary	updated_at
1172	Andrew Lee	110,846	2021-04-11 16:00:00
1172	Andrew Lee	114,859	2021-07-27 16:00:00

```
a = emp.select("employee_id", col("salary")
              .alias("salary_a"))
b = emp.select("employee_id", col("salary")
              .alias("salary_b"))
a.join(b, "employee_id").count() # 342, not 332
```

1172 AFTER THE JOIN: 4 ROWS

employee_id	salary_a	salary_b
1172	110,846	110,846
1172	110,846	114,859
1172	114,859	110,846
1172	114,859	114,859

SECTION 15 · JOINS

CONTINUED

D Range join: salary bands

Any boolean condition works, not only equality. A left join keeps the bad salaries visible: -5,000, 9,999,999 and null fall into no band.

```
rs = emp.filter(col("employee_id")
               .isin(1001, 1009, 1027, 1199,
                    1218))
rs.join(bands,
        (rs.salary >= bands.low) &
        (rs.salary < bands.high),
        "left")
```

RESULT

employee_id	employee_name	salary	band
1001	Danielle Johnson	98,904	B: 80k to 120k
1009	Jeffrey Lawrence	141,212	C: 120k to 200k
1027	Joseph Zuniga	9,999,999	null
1199	Russell Carpenter	null	null
1218	Ms. April Williams	-5000	null

BANDS

low	high	band
0	80,000	A: under 80k
80,000	120,000	B: 80k to 120k
120,000	200,000	C: 120k to 200k

E Self join: colleagues hired on the same day

Alias the deduplicated table (clean, built in Window Functions) twice and match on department and hire date. The $x < y$ condition stops each pair appearing twice and stops people pairing with themselves. These are the ties you will meet again in Window Functions.

```
x, y = clean.alias("x"), clean.alias("y")
x.join(y,
      (col("x.department_id") == col("y.department_id")) &
      (col("x.hire_date") == col("y.hire_date")) &
      (col("x.employee_id") < col("y.employee_id"))) \
.select(col("x.department_id"), col("x.hire_date"),
        col("x.employee_name").alias("employee_1"),
        col("y.employee_name").alias("employee_2"))
```

department_id	hire_date	employee_1	employee_2
102	2020-05-10	Chad Hodges	Selena Duncan
103	2020-10-09	Andrea Williams	Mr. Eric Yu
106	2021-05-22	Raven Gilbert	Troy Sanchez

PATTERN REFERENCE

PATTERN

CODE

01 Inner join

Default join type, keeps only rows that match in both DataFrames

```
result = emp.join(dept, "department_id", "inner")
```

SECTION 15 · JOINS

CONTINUED

PATTERN	CODE
02 Left join Keeps all rows from left, unmatched right columns become null	<pre>result = emp.join(dept, "department_id", "left")</pre>
03 Full outer join Keeps all rows from both sides	<pre>result = emp.join(dept, "department_id", "outer")</pre>
04 Left semi join Efficient IN subquery alternative, keeps only left columns	<pre>result = emp.join(dept, "department_id", "left_semi")</pre>
05 Left anti join Rows in left with no match in right. Not the same as NOT IN: one null key on the right makes NOT IN return no rows, the anti join ignores it	<pre>result = emp.join(dept, "department_id", "left_anti")</pre>
06 Join on multiple columns	<pre># emp_prev: last month's extract with the same columns result = emp.join(emp_prev, ["employee_id", "department_id"], "inner")</pre>
07 Join on columns with different names This keeps both key columns, drop one afterwards	<pre>dept_r = dept.withColumnRenamed("department_id", "dept_id") result = emp.join(dept_r, emp["department_id"] == dept_r["dept_id"], "left") result = result.drop(dept_r["dept_id"])</pre>
08 Broadcast join for small tables Sends the small table to all nodes, eliminates shuffle	<pre>result = emp.join(broadcast(dept), "department_id")</pre>

SECTION 15 · JOINS

CONTINUED

PATTERN

CODE

09 Avoid column ambiguity after join

Rename columns before joining if both DataFrames share names

```
prev = emp_prev.withColumnRenamed("salary", "prev_salary")
result = emp.join(prev.select("employee_id",
                              "prev_salary"), "employee_id", "left")
```

WATCH OUT

`broadcast()`

Spark already broadcasts a side below `spark.sql.autoBroadcastJoinThreshold`, 10 MB by default; `broadcast()` forces it for a larger small side

SECTION 16 · AGGREGATIONS

Aggregations.

08

PATTERNS

01

WATCH OUT

`groupBy`, `agg`, `pivot`, `collect list/set`, `approx distinct`.

PATTERN

CODE

01 Simple count per group

```
result = emp.groupBy("department_id").count()
```

02 Multiple aggregations at once

Always use `.agg()` for multiple calculations, more efficient

```
result = emp.groupBy("department_id").agg(
    spark_sum("salary").alias("total_salary"),
    avg("salary").alias("avg_salary"),
    spark_max("salary").alias("max_salary"),
    spark_min("salary").alias("min_salary"),
    count("employee_id").alias("headcount")
)
```

03 Count distinct values per group

```
result = emp.groupBy("department_id").agg(
    countDistinct("job_title").alias("unique_titles")
)
```

04 Pivot, rows to columns

Specify values explicitly for better performance

```
result = emp.withColumn("hire_year", year("hire_date")) \
    .groupBy("department_id") \
    .pivot("hire_year", [2022, 2023, 2024]).count()
```

SECTION 16 · AGGREGATIONS

CONTINUED

PATTERN

CODE

05 Global aggregation, no groupBy

`collect()` brings result to driver, only for scalar results

```
max_salary = emp.agg(spark_max("salary")).collect()[0][0]
stats = emp.agg(
    mean("salary").alias("mean"),
    stddev("salary").alias("stddev")
)
```

06 Collect list per group

Warning, large lists cause OOM errors, use with care

```
result = emp.groupBy("department_id").agg(
    collect_list("employee_name").alias("employees")
)
```

07 Collect unique values per group

`collect_set` returns unique values only, no duplicates

```
result = emp.groupBy("department_id").agg(
    collect_set("job_title").alias("titles")
)
```

08 Approximate count distinct

Much faster than `countDistinct` on large datasets, uses HyperLogLog

```
result = emp.groupBy("department_id").agg(
    approx_count_distinct("employee_id", 0.05)
    .alias("approx_employees")
)
```

WATCH OUT

`approx_count_distinct`

trades a small bounded error for speed: `rsd` is 0.05 by default, pass a smaller `rsd` for more accuracy

SECTION 17 · WINDOW FUNCTIONS

Window Functions.

09
PATTERNS

01
WATCH OUT

row_number, rank, lead, lag, running total, deduplication.

THE DATA: YOUR EMPLOYEES.CSV, CLEANED, THEN SLICED

1 Keep the latest record per employee

```
w_dedup = Window.partitionBy("employee_id") \
    .orderBy(col("updated_at").desc_nulls_last())
clean = emp \
    .withColumn("rn", row_number().over(w_dedup)) \
    .filter(col("rn") == 1).drop("rn")
# 332 rows in, 327 out
```

1172 BEFORE

employee_id	employee_name	salary	updated_at
1172	Andrew Lee	110,846	2021-04-11 16:00:00
1172	Andrew Lee	114,859	2021-07-27 16:00:00

1172 AFTER

employee_id	employee_name	salary	updated_at
1172	Andrew Lee	114,859	2021-07-27 16:00:00

2 Sales and Product Management hires, July 2020 to June 2021

```
hires = clean.filter(
    col("department_id").isin(103, 106) &
    col("hire_date").between("2020-07-01", "2021-06-30")) \
    .select("department_id", "employee_name",
           "hire_date", "salary")

w = Window.partitionBy("department_id") \
    .orderBy("hire_date")
```

18 people in two partitions. Two pairs joined on the same day: Andrea Williams and Mr. Eric Yu (Sales, 2020-10-09), Raven Gilbert and Troy Sanchez (Product Management, 2021-05-22). Every function below treats those ties differently.

Same-day hires (ties)

Columns that disagree

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

RANKING: SIX FUNCTIONS, ONE WINDOW

```
hires.select("department_id",
             "employee_name", "hire_date",
             row_number().over(w)
             .alias("row_number"),
             rank().over(w).alias("rank"),
             dense_rank().over(w)
             .alias("dense_rank"),
             percent_rank().over(w)
             .alias("percent_rank"),
             cume_dist().over(w)
             .alias("cume_dist"),
             ntile(2).over(w).alias("ntile(2)"))
```

row_number 1, 2, 3, 4 with no repeats. Andrea and Eric are split arbitrarily, so they can swap between runs.

rank ties share a number, then it skips: Sales goes 1, 2, 3, 3, 5.

dense_rank ties share a number, no gap: 1, 2, 3, 3, 4.

percent_rank (rank - 1) / (rows - 1). Andrea: (3 - 1) / 7 = 0.29.

cume_dist share of the partition hired on or before this date: Andrea and Eric both 4 / 8 = 0.5.

ntile(2) two near-equal buckets per department: 4 and 4 in Sales, 5 and 5 in Product Management.

department_id	employee_name	hire_date	row_number	rank	dense_rank	percent_rank	cume_dist	ntile(2)
103	Yvette Huff	2020-09-06	1	1	1	0.0	0.13	1
103	Cheryl Archer	2020-09-23	2	2	2	0.14	0.25	1
103	Andrea Williams	2020-10-09	3	3	3	0.29	0.5	1
103	Mr. Eric Yu	2020-10-09	4	3	3	0.29	0.5	1
103	Brooke Leonard	2020-11-15	5	5	4	0.57	0.63	2
103	Tanya Riley	2020-12-01	6	6	5	0.71	0.75	2
103	Spencer Henderson	2021-01-22	7	7	6	0.86	0.88	2
103	Nicole Suarez	2021-02-05	8	8	7	1.0	1.0	2
106	Diana Marshall	2020-07-31	1	1	1	0.0	0.1	1
106	Robert Soto	2020-08-15	2	2	2	0.11	0.2	1
106	Ms. Caitlin Dodson	2020-09-24	3	3	3	0.22	0.3	1
106	Scott Lewis	2020-10-23	4	4	4	0.33	0.4	1
106	Melissa Jones	2020-12-09	5	5	5	0.44	0.5	1
106	Dr. David Fisher	2021-03-24	6	6	6	0.56	0.6	2
106	Mr. Lee Charles	2021-04-16	7	7	7	0.67	0.7	2
106	Raven Gilbert	2021-05-22	8	8	8	0.78	0.9	2
106	Troy Sanchez	2021-05-22	9	8	8	0.78	0.9	2
106	Diana Smith	2021-05-31	10	10	9	1.0	1.0	2

percent_rank and cume_dist are rounded to 2 decimal places. For a repeatable row_number, add a tiebreaker:

```
orderBy("hire_date", "employee_id").
```

OFFSETS: THE ROW BEFORE OR AFTER

```
lag("hire_date", 1).over(w)      # previous hire, null on the first row
lead("hire_date", 1).over(w)     # next hire, null on the last row
datediff(col("hire_date"), lag("hire_date", 1).over(w)) # days_since_prev
```

department_id	employee_name	hire_date	prev_hire	next_hire	days_since_prev
103	Yvette Huff	2020-09-06	null	2020-09-23	null
103	Cheryl Archer	2020-09-23	2020-09-06	2020-10-09	17
103	Andrea Williams	2020-10-09	2020-09-23	2020-10-09	16
103	Mr. Eric Yu	2020-10-09	2020-10-09	2020-11-15	0
103	Brooke Leonard	2020-11-15	2020-10-09	2020-12-01	37
103	Tanya Riley	2020-12-01	2020-11-15	2021-01-22	16

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

department_id	employee_name	hire_date	prev_hire	next_hire	days_since_prev
103	Spencer Henderson	2021-01-22	2020-12-01	2021-02-05	52
103	Nicole Suarez	2021-02-05	2021-01-22	null	14
106	Diana Marshall	2020-07-31	null	2020-08-15	null
106	Robert Soto	2020-08-15	2020-07-31	2020-09-24	15
106	Ms. Caitlin Dodson	2020-09-24	2020-08-15	2020-10-23	40
106	Scott Lewis	2020-10-23	2020-09-24	2020-12-09	29
106	Melissa Jones	2020-12-09	2020-10-23	2021-03-24	47
106	Dr. David Fisher	2021-03-24	2020-12-09	2021-04-16	105
106	Mr. Lee Charles	2021-04-16	2021-03-24	2021-05-22	23
106	Raven Gilbert	2021-05-22	2021-04-16	2021-05-22	36
106	Troy Sanchez	2021-05-22	2021-05-22	2021-05-31	0
106	Diana Smith	2021-05-31	2021-05-22	null	9

Offsets never cross a partition: Diana Marshall starts Product Management with a null prev_hire, even though Nicole Suarez is printed just above her. Same-day hires show 0 days.

AGGREGATES OVER A WINDOW: TOTALS, SHARES AND RUNNING VALUES

```

wp      = Window.partitionBy("department_id")
wrows   = w.rowsBetween(Window.unboundedPreceding, Window.currentRow)
wrangle = w.rangeBetween(Window.unboundedPreceding, Window.currentRow)

spark_sum("salary").over(wp)                # dept_payroll
col("salary") * 100 / spark_sum("salary").over(wp) # pct_of_payroll
count("*").over(wrows)                       # headcount_rows
count("*").over(wrangle)                     # headcount_range
spark_sum("salary").over(wrows)              # payroll_rows
spark_sum("salary").over(w)                   # payroll_default

```

department_id	employee_name	hire_date	salary	dept_payroll	pct_of_payroll	headcount_rows	headcount_range	payroll_rows	payroll_default
103	Yvette Huff	2020-09-06	74,406	677,921	11.0	1	1	74,406	74,406
103	Cheryl Archer	2020-09-23	67,653	677,921	10.0	2	2	142,059	142,059
103	Andrea Williams	2020-10-09	79,794	677,921	11.8	3	4	221,853	294,059
103	Mr. Eric Yu	2020-10-09	72,206	677,921	10.7	4	4	294,059	294,059
103	Brooke Leonard	2020-11-15	110,151	677,921	16.2	5	5	404,210	404,210
103	Tanya Riley	2020-12-01	85,615	677,921	12.6	6	6	489,825	489,825
103	Spencer Henderson	2021-01-22	96,164	677,921	14.2	7	7	585,989	585,989
103	Nicole Suarez	2021-02-05	91,932	677,921	13.6	8	8	677,921	677,921
106	Diana Marshall	2020-07-31	123,613	1,250,056	9.9	1	1	123,613	123,613
106	Robert Soto	2020-08-15	129,135	1,250,056	10.3	2	2	252,748	252,748
106	Ms. Caitlin Dodson	2020-09-24	164,455	1,250,056	13.2	3	3	417,203	417,203
106	Scott Lewis	2020-10-23	137,645	1,250,056	11.0	4	4	554,848	554,848
106	Melissa Jones	2020-12-09	122,853	1,250,056	9.8	5	5	677,701	677,701
106	Dr. David Fisher	2021-03-24	114,349	1,250,056	9.1	6	6	792,050	792,050
106	Mr. Lee Charles	2021-04-16	115,223	1,250,056	9.2	7	7	907,273	907,273
106	Raven Gilbert	2021-05-22	117,905	1,250,056	9.4	8	9	1,025,178	1,127,351
106	Troy Sanchez	2021-05-22	102,173	1,250,056	8.2	9	9	1,127,351	1,127,351
106	Diana Smith	2021-05-31	122,705	1,250,056	9.8	10	10	1,250,056	1,250,056

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

Read the Andrea Williams row. headcount_rows is 3, headcount_range is 4: a range frame includes every row tied with the current one, so Eric, hired the same day, is counted too. With an orderBy and no explicit frame Spark uses the range frame, which is why payroll_default (294,059) matches the range behaviour, not payroll_rows (221,853). pct_of_payroll is rounded to 1 decimal place.

FRAMES: WHICH ROWS AN AGGREGATE CAN SEE

Frame	Gives you	Rows included
<code>rowsBetween(Window.unboundedPreceding, Window.currentRow)</code>	Running total	Start of partition up to this row, counted in rows
<code>rangeBetween(Window.unboundedPreceding, Window.currentRow)</code>	Running total, ties together	Up to this row and every row tied with it
<code>orderBy(...)</code> with no frame	Same as the range frame	This is why last() looks wrong, see A below
<code>partitionBy(...)</code> with no orderBy	Whole partition	Totals, averages, max per group
<code>rowsBetween(-2, 0)</code>	Trailing 3 rows	This row and the two before it
<code>rowsBetween(-1, 1)</code>	Centred 3 rows	Row before, this row, row after
<code>rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing)</code>	Whole partition, ordered	Needed for last() with an orderBy

THREE PATTERNS WORTH KNOWING, EACH RUN ON YOUR DATA

A last() returns the current row

With an orderBy the default frame ends at the current row, so last() is this row, or the last of its ties: look at Andrea Williams, whose last_default is Mr. Eric Yu. Widen the frame to get the real last hire.

```
first("employee_name").over(w)    # first hire
last("employee_name").over(w)     # stops at this row

full = w.rowsBetween(
    Window.unboundedPreceding,
    Window.unboundedFollowing)
last("employee_name").over(full)   # true last
```

<u>department_id</u>	employee_name	hire_date	first_hire	last_default	last_full_frame
103	Yvette Huff	2020-09-06	Yvette Huff	Yvette Huff	Nicole Suarez
103	Cheryl Archer	2020-09-23	Yvette Huff	Cheryl Archer	Nicole Suarez
103	Andrea Williams	2020-10-09	Yvette Huff	Mr. Eric Yu	Nicole Suarez
103	Mr. Eric Yu	2020-10-09	Yvette Huff	Mr. Eric Yu	Nicole Suarez
103	Brooke Leonard	2020-11-15	Yvette Huff	Brooke Leonard	Nicole Suarez
103	Tanya Riley	2020-12-01	Yvette Huff	Tanya Riley	Nicole Suarez
103	Spencer Henderson	2021-01-22	Yvette Huff	Spencer Henderson	Nicole Suarez
103	Nicole Suarez	2021-02-05	Yvette Huff	Nicole Suarez	Nicole Suarez
106	Diana Marshall	2020-07-31	Diana Marshall	Diana Marshall	Diana Smith
106	Robert Soto	2020-08-15	Diana Marshall	Robert Soto	Diana Smith
106	Ms. Caitlin Dodson	2020-09-24	Diana Marshall	Ms. Caitlin Dodson	Diana Smith
106	Scott Lewis	2020-10-23	Diana Marshall	Scott Lewis	Diana Smith
106	Melissa Jones	2020-12-09	Diana Marshall	Melissa Jones	Diana Smith
106	Dr. David Fisher	2021-03-24	Diana Marshall	Dr. David Fisher	Diana Smith
106	Mr. Lee Charles	2021-04-16	Diana Marshall	Mr. Lee Charles	Diana Smith

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

department_id	employee_name	hire_date	first_hire	last_default	last_full_frame
106	Raven Gilbert	2021-05-22	Diana Marshall	Troy Sanchez	Diana Smith
106	Troy Sanchez	2021-05-22	Diana Marshall	Troy Sanchez	Diana Smith
106	Diana Smith	2021-05-31	Diana Marshall	Diana Smith	Diana Smith

B Top 2 earners per department

On the full cleaned file (317 rows once the null, negative and 9,999,999 salaries are removed). employee_id breaks salary ties so the result is repeatable.

```
valid = clean.filter(col("salary").between(1, 1000000))
wt = Window.partitionBy("department_id") \
    .orderBy(col("salary").desc(), col("employee_id"))
valid.withColumn("rn", row_number().over(wt)) \
    .filter(col("rn") <= 2) \
    .join(dept, "department_id") \
    .select("department_name", "employee_name", "salary", "rn")
```

department_name	employee_name	salary	rn
Cloud Infrastructure	Anne Robinson	147,089	1
Cloud Infrastructure	Mr. Joseph Flores	140,177	2
Cybersecurity	Curtis Barton	148,696	1
Cybersecurity	Hannah Burton	148,196	2
Data & Analytics	Katie Anderson	153,724	1
Data & Analytics	Rodney Owens	145,569	2
Engineering	Mr. Christopher Williams	156,410	1
Engineering	Leslie Adams	155,106	2
Finance	Lisa Keith	128,793	1
Finance	Catherine Dixon	125,093	2
Human Resources	Laura Valencia	84,693	1
Human Resources	Kyle Nolan	81,220	2
Legal & Compliance	James Wilson	130,345	1
Legal & Compliance	Tiffany Warren	120,344	2
Product Management	Ms. Caitlin Dodson	164,455	1
Product Management	Emily Walker	163,191	2
Sales	Charles Ellis	113,734	1
Sales	Alexandra Donovan	110,816	2

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

C Hires per year: running and moving values

The same functions over years. Only 7 rows, so a window with no partitionBy is fine; on a large table it pushes every row to one task.

```
hy = clean.filter(col("hire_date").isNotNull()) \
    .groupBy(year("hire_date").alias("hire_year")) \
    .agg(count("*").alias("hires"))
wy = Window.orderBy("hire_year")
cum = wy.rowsBetween(
    Window.unboundedPreceding, Window.currentRow)

spark_sum("hires").over(cum)    # running_total
avg("hires").over(
    wy.rowsBetween(-2, 0))      # avg_last_3
col("hires") - lag("hires", 1).over(wy)
                                # change
spark_max("hires").over(cum)    # best_so_far
```

hire_year	hires	running_total	avg_last_3	change	best_so_far
2018	38	38	38.0	null	38
2019	34	72	36.0	-4	38
2020	45	117	39.0	11	45
2021	64	181	47.7	19	64
2022	48	229	52.3	-16	64
2023	38	267	50.0	-10	64
2024	50	317	45.3	12	64

PATTERN REFERENCE

PATTERN

CODE

01 Define a window specification

partitionBy groups rows,
orderBy determines
sequence within each group

```
from pyspark.sql.window import Window

w_spec = Window.partitionBy("department_id")
            .orderBy(col("salary").desc())
```

02 Row number

Sequential integer, no ties,
always unique within
partition

```
emp = emp.withColumn("row_num", row_number().over(w_spec))
```

03 Rank with gaps

Tied rows get the same
rank, next rank skips (1, 2, 2,
4)

```
emp = emp.withColumn("rank_val", rank().over(w_spec))
```

SECTION 17 · WINDOW FUNCTIONS

CONTINUED

PATTERN

CODE

04 Dense rank, no gaps

Tied rows get same rank,
next rank is consecutive (1,
2, 2, 3)

```
emp = emp.withColumn("dense_rank", dense_rank()  
    .over(w_spec))
```

05 Lead, next row value

Returns null at partition
boundary unless a default is
set

```
emp = emp.withColumn("next_salary", lead("salary", 1)  
    .over(w_spec))  
emp = emp.withColumn("next_sal_d0", lead("salary", 1, 0)  
    .over(w_spec))
```

06 Lag, previous row value

Returns null at partition
boundary unless a default is
set

```
emp = emp.withColumn("prev_salary", lag("salary", 1)  
    .over(w_spec))  
emp = emp.withColumn("prev_sal_d0", lag("salary", 1, 0)  
    .over(w_spec))
```

07 Running total, cumulative sum

rowsBetween defines the
frame from start of partition
to current row

```
w_running = Window.partitionBy("department_id")  
    .orderBy("hire_date") \  
    .rowsBetween(Window.unboundedPreceding, Window  
    .currentRow)  
emp = emp.withColumn("running_payroll",  
    spark_sum("salary").over(w_running))
```

08 Group average on every row

No orderBy needed for pure
aggregation windows

```
w_dept = Window.partitionBy("department_id")  
emp = emp.withColumn("dept_avg", avg("salary")  
    .over(w_dept))
```

09 Deduplicate keeping latest row

Common pattern for
keeping the most recent
record per key

```
w_dedup = Window.partitionBy("employee_id")  
    .orderBy(col("updated_at").desc())  
emp = emp.withColumn("rn", row_number().over(w_dedup))  
emp = emp.filter(col("rn") == 1).drop("rn")
```

WATCH OUT

frame clause

rowsBetween is safer and more predictable than rangeBetween for most use cases

SECTION 18 · UDFS

UDFs.

06

PATTERNS

02

WATCH OUT

Standard UDF, Pandas UDF, decorator, SQL registration.

PATTERN

CODE

01 Standard Python UDF

Slow, serialises each row to Python. Avoid on large data

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def upper_case_fn(s):
    return s.upper() if s else None

upper_udf = udf(upper_case_fn, StringType())
emp = emp.withColumn("name_upper",
    upper_udf(col("employee_name")))
```

02 Register UDF as decorator

Cleaner syntax, return type annotation sets the output type

```
@udf(returnType=StringType())
def strip_title(s):
    import re
    return re.sub(r"^(Mr|Mrs|Ms|Dr)\.s+", "", s) if s
    else None

emp = emp.withColumn("clean_name",
    strip_title(col("employee_name")))
```

03 Pandas UDF, vectorised and fast

Input and output are pd.Series, much faster than standard UDF

```
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf(DoubleType())
def monthly(salary: pd.Series) -> pd.Series:
    return salary / 12

emp = emp.withColumn("monthly_salary",
    monthly(col("salary")))
```

04 Pandas UDF with multiple input columns

```
@pandas_udf(StringType())
def name_title(name: pd.Series, title: pd.Series) -> pd.Series:
    return name + " (" + title + ")"

emp = emp.withColumn("name_title",
    name_title(col("employee_name"), col("job_title")))
```

05 Register UDF for SQL usage

```
spark.udf.register("sql_upper", upper_case_fn,
    StringType())
emp.createOrReplaceTempView("employees")
names = spark.sql("SELECT sql_upper(employee_name) FROM
    employees")
```

SECTION 18 · UDFS

CONTINUED

06 Prefer built-in functions over UDFs

Native built-in functions run on JVM with no serialisation cost

PREFER

```
# Prefer this
emp = emp.withColumn("name_upper",
    upper(col("employee_name")))
```

AVOID

```
# Over this
emp = emp.withColumn("name_upper",
    upper_udf(col("employee_name")))
```

WATCH OUT

pandas_udf

much faster than a standard UDF because data moves in Arrow batches; prefer a built-in function when one exists

standard UDF

on Spark 3.5+, `udf(..., useArrow=True)` moves rows in Arrow batches and cuts the per-row cost

SECTION 19 · AI FUNCTIONS

AI Functions.

06

PATTERNS

03

WATCH OUT

Classify, extract, similarity, mask, translate, prompts on Databricks and Fabric.

WHICH FUNCTION, ON WHICH PLATFORM

TASK	DATABRICKS (SQL, VIA EXPR)	MICROSOFT FABRIC (DF.AI)
Classify	<code>ai_classify</code>	<code>ai.classify</code>
Extract fields	<code>ai_extract</code>	<code>ai.extract</code>
Sentiment	<code>ai_analyze_sentiment</code>	<code>ai.analyze_sentiment</code>
Summarize	<code>ai_summarize</code>	<code>ai.summarize</code>
Translate	<code>ai_translate</code>	<code>ai.translate</code>
Similarity	<code>ai_similarity</code>	<code>ai.similarity</code>
Fix grammar	<code>ai_fix_grammar</code>	<code>ai.fix_grammar</code>
Mask entities	<code>ai_mask</code>	none: use <code>ai.generate_response</code>
Own prompt	<code>ai_query</code> , <code>ai_gen</code>	<code>ai.generate_response</code>
Embeddings	<code>ai_query</code> on an embedding endpoint	<code>ai.embed</code>
Parse documents	<code>ai_parse_document</code>	<code>input_col_type="path"</code> on any function
Forecast	<code>ai_forecast</code>	none

SECTION 19 · AI FUNCTIONS

CONTINUED

01 Classify job titles into families

Version 2.1 returns a VARIANT with the label under response[0].value; Fabric adds a plain string column

DATABRICKS

```
emp = emp.withColumn("cls", expr("""ai_classify(job_title,
    '["Engineering", "Data", "Sales", "HR", "Finance", "Leadership"]',
    MAP('version', '2.1'))"""))
emp = emp.withColumn("job_family", expr("cls:response[0].value::string"))
```

MICROSOFT FABRIC

```
emp = emp.ai.classify(
    labels=["Engineering", "Data", "Sales", "HR", "Finance", "Leadership"],
    input_col="job_title", output_col="job_family")
```

02 Extract fields from text

Fabric adds one column per label; Databricks returns one VARIANT holding every field

DATABRICKS

```
emp = emp.withColumn("parts", expr(
    """ai_extract(job_title, '["seniority", "function"]')"""))
```

MICROSOFT FABRIC

```
emp = emp.ai.extract(labels=["seniority", "function"], input_col="job_title")
```

03 Find titles similar to a role Semantic, not string matching: higher means closer in meaning. Sort on it to rank**DATABRICKS**

```
emp = emp.withColumn("score", expr("ai_similarity(job_title, 'Data Engineer')"))
```

MICROSOFT FABRIC

```
emp = emp.ai.similarity(input_col="job_title", other="Data Engineer",
    output_col="score")
```

SECTION 19 · AI FUNCTIONS

CONTINUED

04 Mask names in free text

ai_mask is public preview on Databricks. Fabric has no mask function, so use a template prompt

```
emp = emp.withColumn("bio", concat_ws(" ", col("employee_name"), lit("works as"),
    col("job_title")))
```

DATABRICKS

```
emp = emp.withColumn("bio_masked", expr("ai_mask(bio, array('person'))"))
```

MICROSOFT FABRIC

```
emp = emp.ai.generate_response(
    prompt="Replace every person's name in {bio} with [MASKED]. Return only the
    text.",
    is_prompt_template=True, output_col="bio_masked")
```

05 Translate department names

ai_translate is public preview on Databricks and takes a language code or name

DATABRICKS

```
dept = dept.withColumn("name_es", expr("ai_translate(department_name, 'es')"))
```

MICROSOFT FABRIC

```
dept = dept.ai.translate(to_lang="spanish", input_col="department_name",
    output_col="name_es")
```

06 Run your own prompt on every row

failOnError => false returns a struct of response and errorMessage, so one bad row does not fail the job

DATABRICKS

```
emp = emp.withColumn("welcome", expr("""
    ai_query('system.ai.meta-llama-3-3-70b-instruct',
        concat('Write a one-line welcome note for ', employee_name,
            ', our new ', job_title),
        failOnError => false)"""))
```

MICROSOFT FABRIC

```
emp = emp.ai.generate_response(
    prompt="Write a one-line welcome note for {employee_name}, our new
    {job_title}",
    is_prompt_template=True, output_col="welcome")
```

SECTION 19 · AI FUNCTIONS

CONTINUED

ON DATABRICKS AND FABRIC

DATABRICKS

SQL functions, called from PySpark with `expr()` or `spark.sql`. Needs Databricks Runtime 15.4 LTS or later (18.2+ recommended), a supported region, and not a Classic SQL warehouse. `ai_query` needs CAN QUERY on the endpoint.

MICROSOFT FABRIC

DataFrame methods under `df.ai`, nothing to install on Runtime 1.3 or later. Needs a paid F2+ or P capacity and the tenant switch for Copilot and Azure OpenAI features. The default model is gpt-5-mini; change it with

```
aifunc.default_conf.set_deployment_name()  
from synapse.ml.spark.aifunc.
```

WATCH OUT**cost**

every row is a model call: test on `emp.limit(20)`, then run the full table in one query, not hand-made batches

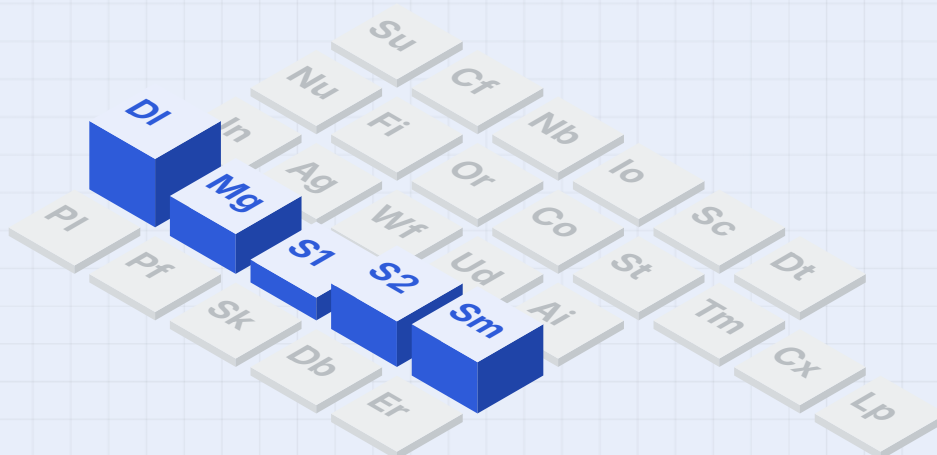
repeatability

a rerun can give different answers: write results to a table once and review them before use

personal data

rows go to a model endpoint; Fabric says it does not log or store them, but check your data policy first

CHAPTER 04 OF 05



Delta Lake.

Store and version data, update it with MERGE, keep history with slowly changing dimensions, then stream into it.

05

SECTIONS

40

PATTERNS

13

WATCH OUT

IN THIS CHAPTER

20	Delta & Time Travel	51
21	Delta: Advanced MERGE	53
22	SCD Type 1	55
23	SCD Type 2	56
24	Streaming	58

SECTION 20 · DELTA & TIME TRAVEL

Delta & Time Travel.

12
PATTERNS

03
WATCH OUT

Read, write, time travel, restore, vacuum, optimize, Z-order.

PATTERN	CODE
01 Read Delta table Delta provides ACID transactions, schema enforcement and versioning	<pre>from delta.tables import DeltaTable emp = spark.read.format("delta").load("path/to/employees_delta")</pre>
02 Write to Delta	<pre>emp.write.format("delta").mode("overwrite").save("path/to/employees_delta") emp.write.format("delta").mode("append").save("path/to/employees_delta")</pre>
03 Read a specific version Version numbers start at 0 and increment on every write	<pre>emp_v5 = spark.read.format("delta") \ .option("versionAsOf", 5) \ .load("path/to/employees_delta")</pre>
04 Read as of a timestamp Useful for point-in-time recovery or auditing	<pre>emp_asof = spark.read.format("delta") \ .option("timestampAsOf", "2025-03-01 00:00:00") \ .load("path/to/employees_delta")</pre>
05 View table history Shows every operation, version, timestamp and user	<pre>emp_tbl = DeltaTable.forPath(spark, "path/to/employees_delta") emp_tbl.history().show(truncate=False)</pre>
06 Restore to a version or timestamp Delta 1.2+ Overwrites the current table state, use with caution	<pre>emp_tbl.restoreToVersion(5) emp_tbl.restoreToTimestamp("2025-03-01")</pre>
07 Vacuum, clean up old files Default retention is 7 days (168 hours), do not go lower without care	<pre>emp_tbl.vacuum(168) # retention in hours spark.sql("VACUUM delta.`path/to/employees_delta` RETAIN 168 HOURS DRY RUN") # preview first</pre>
08 Optimise, compact small files Delta 2.0+ Run periodically to keep read performance fast	<pre>emp_tbl.optimize().executeCompaction() emp_tbl.optimize().executeZOrderBy("department_id", "hire_date")</pre>

SECTION 20 · DELTA & TIME TRAVEL

CONTINUED

PATTERN

CODE

09 Replace one slice with replaceWhere

Overwrites only the rows that match the predicate; the rest of the table is untouched

```
emp.filter(col("department_id") == 101) \
    .write.format("delta").mode("overwrite") \
    .option("replaceWhere", "department_id = 101") \
    .save("path/to/employees_delta")
```

10 Add new columns on write

Without mergeSchema, a write with an extra column fails schema enforcement

```
emp.withColumn("bonus", col("salary") * 0.1) \
    .write.format("delta").mode("append") \
    .option("mergeSchema", "true") \
    .save("path/to/employees_delta")
```

11 Read changes with Change Data Feed

Delta 2.0+

Only changes made after it is enabled are recorded. Rows carry _change_type, _commit_version and _commit_timestamp

```
spark.sql("ALTER TABLE hr.employees "
          "SET TBLPROPERTIES (delta.enableChangeDataFeed = "
          "true)")
changes = spark.read.format("delta") \
    .option("readChangeFeed", "true") \
    .option("startingVersion", 5) \
    .table("hr.employees")
```

12 Enforce a rule with a CHECK constraint

Adding it fails while any existing row breaks the rule: your file has one salary of -5000

```
spark.sql("""
    ALTER TABLE hr.employees
    ADD CONSTRAINT salary_positive CHECK (salary > 0)
    """)
```

SECTION 20 · DELTA & TIME TRAVEL

CONTINUED

ON DATABRICKS AND FABRIC

DATABRICKS

Liquid clustering replaces partitioning and ZORDER for new tables (GA on 15.4 LTS and later). Run OPTIMIZE to cluster the data.

```
emp.write.clusterBy("department_id").saveAsTable("main.hr.employees")
```

MICROSOFT FABRIC

V-Order is off by default in new workspaces. Turn it on for a write, a session, or a table; OPTIMIZE can apply it.

```
emp.write.format("delta").option("parquet.vorder.enabled", "true") \
    .saveAsTable("employees")
spark.conf.set("spark.sql.parquet.vorder.default", "true")
```

WATCH OUT

vacuum()

never vacuum below 7 days if concurrent readers are active, data loss risk

optimize()

run after bulk loads to compact small files and keep scan performance high

merge()

atomic upsert in one pass, more efficient than separate delete and insert operations

SECTION 21 · DELTA: ADVANCED MERGE

Delta: Advanced MERGE.

07
PATTERNS

03
WATCH OUT

Upsert, delete, full sync, insert-only, schema evolution.

01 Basic upsert, update and insert Most common MERGE pattern, handles new and changed rows

```
from delta.tables import DeltaTable

emp_tbl = DeltaTable.forPath(spark,
    "path/to/employees_delta") ❶

emp_tbl.alias("tgt").merge( ❷
    updates_df.alias("src"),
    "tgt.employee_id = src.employee_id" ❸
).whenMatchedUpdateAll() \ ❹
.whenNotMatchedInsertAll() \ ❺
.execute() ❻
```

- ❶ Load the target as a DeltaTable handle.
- ❷ Alias both sides so every condition reads tgt.* and src.*.
- ❸ The match condition. Keep it on the business key.
- ❹ Matched rows: overwrite every column from the source.
- ❺ Source rows with no match: insert them.
- ❻ Nothing runs until execute().

SECTION 21 · DELTA: ADVANCED MERGE

CONTINUED

VARIATIONS: SAME MERGE(), CHANGE THE CLAUSES AFTER IT

updates_df holds new and changed rows, one per employee_id; leavers_df holds rows to remove. For a conditional update that only rewrites changed rows, see Section 22, pattern 03.

02 Delete matched rows

Use for hard deletes or expiring records

```
leavers_df.alias("src"),
"tgt.employee_id =
src.employee_id"
).whenMatchedDelete() \
.execute()
```

03 Full sync: update, insert and delete

Delta 2.3+

Also deletes target rows that are missing from the source

```
).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.whenNotMatchedBySourceDelete() \
.execute()
```

04 Insert only, skip existing rows

Inserts only keys the table does not have yet, so a re-run adds nothing

```
updates_df.alias("src"),
"tgt.employee_id =
src.employee_id"
).whenNotMatchedInsertAll() \
.execute()
```

05 Schema evolution on MERGE

Delta 3.2+

withSchemaEvolution() lets new columns in the source be added to the target

```
).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.withSchemaEvolution() \
.execute()
```

06 Overwrite schema entirely

Use when target schema must be replaced, not extended

```
emp.write.format("delta") \
.mode("overwrite") \
.option("overwriteSchema",
"true") \
.save("path/to/employees_delta")
```

07 MERGE using SQL syntax

Equivalent to the DataFrame API, useful in notebooks

```
spark.sql("""
MERGE INTO employees_delta AS tgt
USING employee_updates AS src
ON tgt.employee_id =
src.employee_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
""")
```

WATCH OUT

whenMatchedUpdate

use a condition to skip unchanged rows, avoids rewriting and speeds up MERGE significantly

overwriteSchema

only use when fully replacing the schema, it drops columns not in the new DataFrame

NOT MATCHED BY SOURCE

deletes or updates target rows with no match in the source; whenNotMatchedBySource needs Delta 2.3+

SECTION 22 · SCD TYPE 1

SCD Type 1.

04
PATTERNS

03
WATCH OUT

MERGE overwrite, dedupe the source, null-safe checks.

PATTERN	CODE
01 Deduplicate the source first Keep the latest row per key: 327 of your 332 rows	<pre>w = Window.partitionBy("employee_id") \ .orderBy(col("updated_at").desc()) src = emp.withColumn("rn", row_number().over(w)) \ .filter(col("rn") == 1).drop("rn")</pre>
02 Overwrite matches, insert new keys No history: the old salary is gone once this runs	<pre>from delta.tables import DeltaTable dim = DeltaTable.forName(spark, "hr.dim_employee") dim.alias("t").merge(src.alias("s"), "t.employee_id = s.employee_id").whenMatchedUpdateAll() \ .whenNotMatchedInsertAll() \ .execute()</pre>
03 Update only when a tracked column changed Unchanged rows are not rewritten. <code><=></code> treats a change to or from null as a change	<pre>dim.alias("t").merge(src.alias("s"), "t.employee_id = s.employee_id").whenMatchedUpdate(condition="NOT (t.salary <=> s.salary) OR " "NOT (t.job_title <=> s.job_title)", set={"salary": "s.salary", "job_title": "s.job_title", "updated_at": "s.updated_at"}).whenNotMatchedInsertAll() \ .execute()</pre>
04 Type 1 without Delta, on Parquet Write to a new path, then switch readers to it	<pre>dim_df = spark.read.parquet("path/to/dim_employee") unchanged = dim_df.join(src, "employee_id", "left_anti") new_dim = unchanged.unionByName(src) new_dim.write.mode("overwrite") \ .parquet("path/to/dim_employee_v2")</pre>

WATCH OUT

Type 1 or Type 2

Type 1 shows today's value for all of history. Use Type 2 when reports must show what was true at the time

duplicate keys

MERGE fails when two source rows match the same target row, so deduplicate first (pattern 01)

same path

overwriting the Parquet path you are reading deletes its files before the job fails: on Spark 4.0.1 no files were left

SECTION 23 · SCD TYPE 2

SCD Type 2.

08
PATTERNS

01
WATCH OUT

Full 8-step slowly changing dimension type 2 pattern.

EIGHT STEPS, RUN IN ORDER

01

Filter active records in target

Only active rows can change; closed history is carried over untouched in step 08

```
# dim_emp: employee_id, salary, start_date,
    end_date, is_active
active  = dim_emp.filter(col("is_active") == "Y")
history = dim_emp.filter(col("is_active") == "N")
```

02

Prefix source columns to avoid ambiguity

The source needs one row per key, or a repeated id gets two active rows

```
# clean: one row per employee_id, built in Window
    Functions
src = clean.select(
    *[col(c).alias(f"src_{c}") for c in
      ("employee_id", "salary")]
)
```

03

Full outer join source with active target

Finds changed and brand-new keys in one join; keys missing from the source stay active

```
joined = active.join(src, col("employee_id") ==
    col("src_employee_id"), "full")
```

04

Identify changed records

eqNullSafe also catches a change to or from null; != would skip it

```
updates = joined.filter(
    col("employee_id").isNotNull() &
    col("src_employee_id").isNotNull() &
    ~col("salary").eqNullSafe(col("src_salary"))
)
```

05

Identify brand new records

No match in target means id is null from the left side

```
new_recs = joined.filter(col("employee_id")
    .isNull())
```

06

Close old records

Set is_active to N and stamp end_date for changed records

```
closed = updates.select("employee_id", "salary",
    "start_date") \
    .withColumn("end_date", current_date()) \
    .withColumn("is_active", lit("N"))
```

SECTION 23 · SCD TYPE 2

CONTINUED

07

Prepare new active records

Combine updates and net-new records into fresh active rows

```
active_new = updates.unionByName(new_recs) \
    .select("src_employee_id", "src_salary") \
    .toDF("employee_id", "salary") \
    .withColumn("start_date", current_date()) \
    .withColumn("end_date", lit(None)) \
    .cast("date") \
    .withColumn("is_active", lit("Y"))
```

08

Union all parts into final table

Old history + unchanged active rows + rows just closed + new active rows

```
unchanged = active.join(updates
    .select("employee_id", "employee_id",
        "left_anti")
final_df = history.unionByName(unchanged) \
    .unionByName(closed) \
    .unionByName(active_new)
```

WATCH OUT

left_anti

anti-join only the active rows: run on the whole dimension it would also drop the closed history of every changed employee

SECTION 24 · STREAMING

Streaming.

09
PATTERNS

03
WATCH OUT

readStream, writeStream, triggers, watermark, Kafka.

READ IT TOP TO BOTTOM: SOURCE, TRANSFORM, SINK

1 · SOURCE

01 Read a stream from files

Schema must be defined explicitly for file-based sources

```
emp_stream = spark.readStream \
    .schema(schema) \
    .option("header", "true") \
    .option("maxFilesPerTrigger", 1) \
    .csv("path/to/incoming_employees/")
```

02 Read a stream from Kafka

```
emp_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "host:9092") \
    .option("subscribe", "employee_updates") \
    .load()
```



2 · TRANSFORM

03 Apply transformations

Streaming DataFrames support the same API as batch DataFrames

```
processed = emp_stream \
    .filter(col("salary") > 0) \
    .withColumn("ingested_at", current_timestamp())
```

04 Watermarking for late data

Drops data older than the threshold from stateful aggregations

```
updates_per_window = emp_stream \
    .withWatermark("updated_at", "10 minutes") \
    .groupBy(window("updated_at", "5 minutes")) \
    .count()
```



SECTION 24 · STREAMING

CONTINUED

3 · SINK

05 Write stream to console

For debugging only, never in production. `awaitTermination()` blocks until the stream stops: use it in scripts, not notebook cells

```
query = processed.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()
query.awaitTermination()
```

06 Write stream to Delta

Checkpointing is mandatory for fault-tolerant restarts

```
query = processed.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "path/to/checkpoints/employees") \
    .start("path/to/employees_stream")
```

07 MERGE each micro-batch with foreachBatch

The standard way to upsert from a stream. Deduplicate batch_df first if a key can repeat

```
from delta.tables import DeltaTable
def upsert(batch_df, batch_id):
    DeltaTable.forName(spark, "hr.dim_employee").alias("t").merge(
        batch_df.alias("s"), "t.employee_id = s.employee_id"
    ).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

emp_stream.writeStream.foreachBatch(upsert) \
    .option("checkpointLocation", "path/to/checkpoints/dim_employee") \
    .trigger(availableNow=True).start()
```



CONTROLS

08 Trigger modes availableNow: Spark 3.3+

`availableNow` processes all backlog then stops, behaves like a batch job

```
query = processed.writeStream.format("console") \
    .trigger(processingTime="10 seconds").start()

query = processed.writeStream.format("console") \
    .trigger(availableNow=True).start()
```

SECTION 24 · STREAMING

CONTINUED

09 Monitor a running query

```
print(query.status)
print(query.lastProgress)
query.stop()
```

ON DATABRICKS AND FABRIC

DATABRICKS

Auto Loader (format cloudFiles) is the recommended file source: it tracks new files and infers and evolves the schema.

```
spark.readStream.format("cloudFiles") \
    .option("cloudFiles.format", "csv") \
    .option("cloudFiles.schemaLocation", "/Volumes/main/hr/raw/_schema") \
    .load("/Volumes/main/hr/raw/incoming/")
```

MICROSOFT FABRIC

Structured Streaming runs as shown. Keep checkpoints in the lakehouse, for example Files/checkpoints/employees. The native execution engine does not accelerate streaming; those queries run on standard Spark.

WATCH OUT

availableNow

use instead of a full batch job when you want streaming semantics with batch scheduling

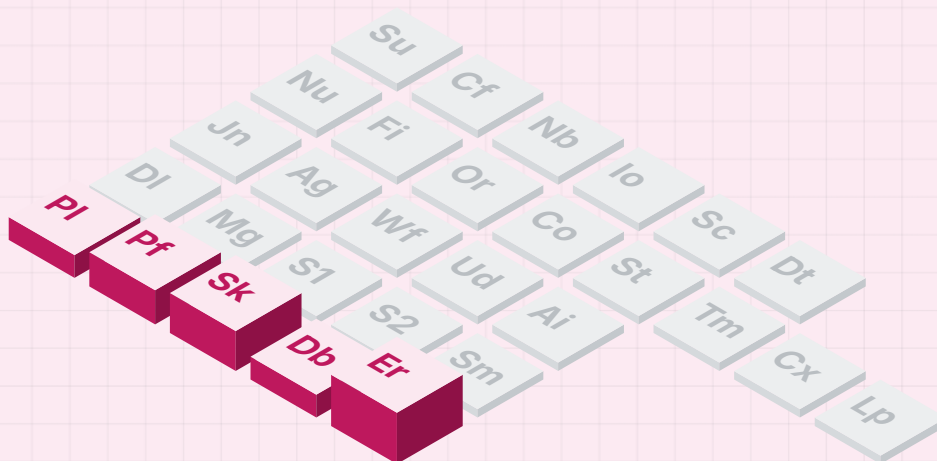
watermark

required for stateful operations on event time, prevents unbounded state growth

outputMode

append adds new rows only, complete replaces the full result, update sends changed rows

CHAPTER 05 OF 05



Tune & Troubleshoot.

Read the plan, fix slow jobs and skew, debug, and handle errors so bad data stops where it starts.

05

SECTIONS

30

PATTERNS

08

WATCH OUT

IN THIS CHAPTER

25	Spark Plans	62
26	Performance & Optimisation	64
27	Performance: Skew Handling	66
28	Debugging	69
29	Error Handling & Logging	70

SECTION 25 · SPARK PLANS

Spark Plans.

04
PATTERNS

02
WATCH OUT

explain modes, shuffles, join strategy, pushdown, adaptive plans.

FIVE WAYS TO PRINT A PLAN

CALL	SHOWS
<code>explain()</code>	the physical plan only
<code>explain("extended")</code>	parsed, analyzed and optimized logical plans, then the physical plan
<code>explain("formatted")</code>	a short physical outline, then details for each numbered node
<code>explain("cost")</code>	the optimized logical plan with size estimates
<code>explain("codegen")</code>	the Java code Spark generates

A REAL PLAN FROM YOUR DATA, READ FROM THE BOTTOM UP

```
q = (emp.filter(col("salary") > 100000)
      .join(dept, "department_id")
      .groupBy("department_name").count())
q.explain()
```

```
AdaptiveSparkPlan isFinalPlan=false ❶
+- HashAggregate(keys=[department_name], functions=[count(1)])
   +- Exchange hashpartitioning(department_name, 200) ❷
      +- HashAggregate(keys=[department_name], functions=[partial_count(1)])
         +- Project [department_name]
            +- BroadcastHashJoin [department_id], [department_id], Inner, BuildRight ❸
               :- Project [department_id]
                  : +- Filter ((isnotnull(salary) AND (salary > 100000)) AND ...)
                  :    +- FileScan parquet [department_id,salary]
                  :       PushedFilters: [..., GreaterThan(salary,100000), ...] ❹
            +- BroadcastExchange HashedRelationBroadcastMode(...) ❺
               +- Filter isnotnull(department_id)
                  +- FileScan parquet [department_id,department_name]
```

- ❶ AQE is on, so the plan can still change while it runs.
- ❷ Exchange is a shuffle: rows move between executors. Count them.
- ❸ departments is under 10 MB, so it is copied to every executor and the join needs no shuffle.
- ❹ The salary filter is applied while reading Parquet, so fewer rows leave the files.
- ❺ The small side being sent to the executors.

SECTION 25 · SPARK PLANS

CONTINUED

PATTERN	CODE
01 Print the plan Look for unexpected SortMergeJoins, missing filters and skewed partitions	<pre>emp.explain(mode="extended") # full logical and physical plan emp.explain(mode="formatted") # human-readable formatted plan</pre>
02 Check the join strategy On your data, turning broadcast off adds an Exchange and a Sort on each side	<pre>spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1") joined = emp.join(dept, "department_id") # after the change joined.explain() # SortMergeJoin spark.conf.unset("spark.sql.autoBroadcastJoinThreshold")</pre>
03 Check filter pushdown PushedFilters in the FileScan means Parquet skips rows while reading	<pre>emp_pq = spark.read.parquet("path/to/employees") emp_pq.filter(col("salary") > 100000).explain() # FileScan parquet ... PushedFilters: [IsNotNull(salary), # GreaterThan(salary,100000)]</pre>
04 See the final adaptive plan Before an action AQE shows isFinalPlan=false; after it, the plan it really ran	<pre>result = emp.join(dept, "department_id") .groupBy("department_name").count() result.collect() result.explain() # AdaptiveSparkPlan isFinalPlan=true</pre>

WATCH OUT

cached plans

the plan is fixed the first time a DataFrame is explained
 or run: after spark.conf.set, rebuild it before
 explain() (checked on 3.5 and 4.0)

explain() is free

it plans the query without running it, so it is safe on any
 size of data

SECTION 26 · PERFORMANCE & OPTIMISATION

Performance & Optimisation.

06
PATTERNS

01
WATCH OUT

Cache, persist, repartition, coalesce, shuffle tuning.

PATTERN	CODE
01 Cache a DataFrame Lazy, data is stored in memory only after the first action is triggered	<pre>emp.cache() emp.count() # trigger the cache emp.unpersist() # always unpersist when done</pre>
02 Persist to memory and disk Prevents OOM when the DataFrame is too large for memory alone	<pre>from pyspark import StorageLevel emp.persist(StorageLevel.MEMORY_AND_DISK) emp.unpersist()</pre>
03 Repartition, increase parallelism Full shuffle into evenly sized partitions, to raise parallelism before a heavy step. It does not split a hot key	<pre>emp_repart = emp.repartition(200) emp_repart = emp.repartition(200, "department_id") # partition by column</pre>
04 Coalesce, reduce output files No shuffle, only merges existing partitions. coalesce(1) runs the last stage as one task; repartition(1) adds a shuffle but keeps the work before it parallel	<pre>emp_one = emp.coalesce(1) # single output file emp_ten = emp.coalesce(10) # reduce to 10 files</pre>
05 Tune shuffle partitions Default is 200. AQE merges small partitions after the shuffle, so there is no fixed rule per core: raise it for very large shuffles and check task sizes in the Spark UI	<pre>spark.conf.set("spark.sql.shuffle.partitions", "400")</pre>

SECTION 26 · PERFORMANCE & OPTIMISATION

CONTINUED

06 Filter and select early Spark pushes a filter below a join on its own, but not below a cache

PREFER

```
# Good, cache only the rows you need
eng = emp.filter(col("department_id")
    == 101).cache()
eng.count()
```

AVOID

```
# Bad, caches all of emp, then
    filters
eng = emp.cache()
    .filter(col("department_id") ==
    101)
```

ON DATABRICKS AND FABRIC

DATABRICKS

Photon speeds up DataFrame and SQL work but does not run Python UDFs, RDD or Dataset APIs; queries that use them switch to standard Spark. Built-in functions keep a query on Photon.

MICROSOFT FABRIC

The native execution engine (Velox and Gluten) runs Parquet and Delta work natively and falls back to standard Spark automatically for unsupported operators. Disable it for a single query with:

```
spark.conf.set("spark.native.enabled", "false")
```

WATCH OUT

shuffle.partitions

reduce below 200 for small data, too many partitions causes overhead from tiny tasks

SECTION 27 · PERFORMANCE: SKEW HANDLING

Performance: Skew Handling.

07
PATTERNS

03
WATCH OUT

AQE skew join, salting, hot key isolation, repartition.

WHICH FIX TO REACH FOR, IN ORDER

SYMPTOM A join stage has one task running far longer than the rest. Confirm it first with

- 01** Detect skewed partitions **02** Detect skewed join keys

- 1** Is AQE available on this runtime? **YES >** **03** Enable AQE skew join optimisation

↓ NO, OR STILL SLOW

- 2** Does the small side fit in executor memory? **YES >** **04** Broadcast the small side

↓ NO

- 3** Does a single known key dominate? **YES >** **05** Isolate and handle the hot key separately

↓ NO, SKEW IS SPREAD OR SEVERE

- 4** Spread the hot keys yourself **THEN ,** **06** Salt skewed join key manually

Only for two large tables: **07** Repartition on join key before joining

01 Detect skewed partitions

Any partition with far more rows than average is skewed

```
partition_counts = emp.rdd.glom().map(len).collect()
print(f"min: {min(partition_counts)}")
print(f"max: {max(partition_counts)}")
print(f"avg: {sum(partition_counts)/len(partition_counts):.0f}")
```

02 Detect skewed join keys

Keys with millions of rows dominate shuffle and cause straggler tasks

```
emp.groupBy("department_id") \
    .count() \
    .orderBy(col("count").desc()) \
    .show(10)
```

SECTION 27 · PERFORMANCE: SKEW HANDLING

CONTINUED

03 Enable AQE skew join optimisation

AQE splits large partitions automatically at runtime, no code change needed. On by default since Spark 3.2

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# Tune the threshold (default 256MB)
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
               "268435456")
```

04 Broadcast the small side

Eliminates shuffle entirely when one side is small enough

```
# Increase broadcast threshold if needed (default 10MB)
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "52428800") # 50MB

result = emp.join(broadcast(dept), "department_id")
```

05 Isolate and handle the hot key separately

Split processing so the hot key does not slow down all other keys

```
hot_key = 101 # Engineering, the largest department

# Process hot key with broadcast
emp_hot = emp.filter(col("department_id") == hot_key)
result_hot = emp_hot.join(broadcast(dept), "department_id")

# Process rest normally
emp_rest = emp.filter(col("department_id") != hot_key)
result_rest = emp_rest.join(dept, "department_id")

result = result_hot.unionByName(result_rest)
```

SECTION 27 · PERFORMANCE: SKEW HANDLING

CONTINUED

06 Salt skewed join key manually

Distributes hot keys across multiple partitions by adding a random suffix

```
import random

# Add salt to the large DataFrame
num_buckets = 10
emp_salted = emp.withColumn("salt",
    (rand() * num_buckets).cast("int")
)
emp_salted = emp_salted.withColumn("salted_key",
    concat(col("department_id"), lit("_"), col("salt"))
)

# Explode the small DataFrame to match all salt values
dept_salted = dept.withColumn("salt_range",
    array([lit(i) for i in range(num_buckets)])
)
dept_salted = dept_salted.withColumn("salt", explode(col("salt_range")))
dept_salted = dept_salted.withColumn("salted_key",
    concat(col("department_id"), lit("_"), col("salt"))
)

# Join on salted key
result = emp_salted.join(dept_salted.drop("department_id", "salt"), "salted_key",
    "left")
```

07 Repartition on join key before joining

Pays off only when neither side can be broadcast: the join then reuses this shuffle. Otherwise it adds two. It does not split a hot key

```
emp_p = emp.repartition(200, "department_id")
dept_p = dept.repartition(200, "department_id")
result = emp_p.join(dept_p, "department_id")
```

WATCH OUT

AQE skew join

on by default since Spark 3.2 and handles most skew with no code change; check it has not been turned off

broadcast

the simplest fix when the small side fits in memory; try it before salting

salting

use when AQE is not available or skew is severe, adds complexity so only use when needed

SECTION 28 · DEBUGGING

Debugging.

04
PATTERNS

01
WATCH OUT

show without truncation, describe, clearCache.

PATTERN	CODE
01 Show data without truncation	<pre>emp.show(n=20, truncate=False) emp.show(n=5, truncate=100)</pre>
02 Profile basic statistics describe() works on numeric and string columns	<pre>emp.describe().show() emp.describe("salary").show()</pre>
03 Clear cache Call when iterative development fills up executor memory	<pre>spark.catalog.clearCache()</pre>
04 Bring results into Python Both pull rows onto the driver: limit first, or a large table can run it out of memory	<pre>names = [r.employee_name for r in emp .select("employee_name").limit(100).collect()] pdf = emp.limit(1000).toPandas()</pre>

WATCH OUT

describe()
quick sanity check on numeric ranges, catches nulls, zeros and unexpected outliers fast

SECTION 29 · ERROR HANDLING & LOGGING

Error Handling & Logging.

09
PATTERNS

01
WATCH OUT

badRecordsPath, try/except, logging, assert_count, schema validation.

PATTERN

CODE

01 Handle bad records in CSV and JSON reads

PERMISSIVE mode stores corrupt records in a special column

```
schema_c = StructType(schema.fields +
    [StructField("_corrupt_record", StringType(), True)])
emp = spark.read \
    .option("header", "true") \
    .option("mode", "PERMISSIVE") \
    .option("columnNameOfCorruptRecord",
        "_corrupt_record") \
    .schema(schema_c) \
    .csv("employees.csv")

# Inspect bad rows (cache first: Spark blocks queries on
# the corrupt column alone)
emp.cache()
bad_rows = emp.filter(col("_corrupt_record").isNotNull())
bad_rows.show(truncate=False)
```

02 Drop malformed records

Silently skips rows that do not match the schema

```
emp = spark.read \
    .option("header", "true") \
    .option("mode", "DROPMALFORMED") \
    .schema(schema) \
    .csv("employees.csv")
```

03 Route bad records to a separate path

Saves corrupt records to a folder for inspection without failing the job. Databricks only: open-source Spark ignores this option

```
emp = spark.read \
    .option("badRecordsPath", "path/to/bad_records/") \
    .schema(schema) \
    .json("employees.json")
```

04 Try/except around a Spark action

Wrap actions: transformations are lazy, so runtime failures surface when an action such as `write()` runs

```
try:
    emp.write.mode("overwrite")
    .parquet("path/to/employees")
    print("Write succeeded")
except Exception as e:
    print(f"Write failed: {e}")
    raise
```

SECTION 29 · ERROR HANDLING & LOGGING

CONTINUED

PATTERN

CODE

05 Set up Python logging in a pipeline

Use logging instead of `print()` for structured, level-filtered output

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%asctime)s %(levelname)s %(message)s"
)

logger = logging.getLogger(__name__)

logger.info(f"Row count before filter: {emp.count()}")
logger.warning("Null salaries detected")
logger.error("Schema mismatch, aborting pipeline")
```

06 Assert row count at pipeline checkpoints

Fail fast if data volumes look wrong rather than propagating bad data

```
def assert_count(df, min_rows, label=""):
    count = df.count()
    if count < min_rows:
        raise ValueError(
            f"{label}: expected >= {min_rows} rows, got {count}"
        )
    logger.info(f"{label}: {count} rows passed")
    return df

emp = assert_count(emp, 300, "after load")
```

07 Validate schema matches expected

Catch schema drift early before it breaks downstream transforms

```
def assert_schema(df, expected_schema, label=""):
    actual = set((f.name, str(f.dataType)) for f in df
                 .schema.fields)
    expected = set((f.name, str(f.dataType)) for f in
                   expected_schema.fields)
    missing = expected - actual
    if missing:
        raise ValueError(f"{label}: missing fields {missing}")
    logger.info(f"{label}: schema validated")

assert_schema(emp, schema, "raw ingestion")
```

SECTION 29 · ERROR HANDLING & LOGGING

CONTINUED

PATTERN

CODE

08 Graceful pipeline with finally block

For scripts and jobs: the session stops even on failure. In a notebook, leave `spark.stop()` out of the finally block

```
try:
    emp = spark.read.parquet("path/to/employees")
    emp = emp.filter(col("salary") > 0)
    emp.write.mode("overwrite").parquet("path/to/employees_clean")
except Exception as e:
    logger.error(f"Pipeline failed: {e}")
    raise
finally:
    spark.stop()
    logger.info("Spark session stopped")
```

09 Unit-test a transformation

Spark 3.5+

Raises `DIFFERENT_ROWS` with a row-by-row diff when the results differ

```
from pyspark.testing import assertDataFrameEqual
expected = spark.createDataFrame(
    [(1001, "Danielle Johnson")], "employee_id int, employee_name string")
actual = emp.filter(col("employee_id") == 1001) \
    .select("employee_id", "employee_name")
assertDataFrameEqual(actual, expected)
```

ON DATABRICKS AND FABRIC

DATABRICKS

`badRecordsPath` works here: bad rows are written to that path for later inspection.

MICROSOFT FABRIC

No `badRecordsPath`: use `PERMISSIVE` with a `_corrupt_record` column (pattern 01).

WATCH OUT

try/except

runtime errors surface at the action, but analysis errors such as a missing column are raised as soon as the transformation is defined

APPENDIX A · COMMON ERRORS

09

ERRORS

Common errors and fixes.

Each error below was triggered on Spark 4.0.1 with your employees.csv, and each fix was run to confirm it works. Messages are shortened.

ERROR	WHY IT HAPPENS	FIX
[AMBIGUOUS_REFERENCE] Reference `salary` is ambiguous, could be: [`salary`, `salary`]. SQLSTATE: 42704	Selecting a column that exists on both sides of a join.	<pre>emp.alias("e").join(emp.alias("m"), col("e.department_id") == col("m.department_id")) \ .select(col("e.salary"))</pre>
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column, variable, or function parameter with name `salry` cannot be resolved. Did you mean one of the following? [`sal...	A typo, or a column dropped earlier. Raised as soon as the transformation is defined, before any action.	<pre># Spark also suggests the closest names print(emp.columns)</pre>
[CAST_INVALID_INPUT] The value 'Danielle Johnson' of the type "STRING" cannot be cast to "INT" because it is malformed. Correct the value as ...	ANSI mode (the Spark 4 default) refuses a value that cannot be converted.	<pre># Spark 4.0+: null instead of an error col("employee_name").try_cast("int")</pre>
[DIVIDE_BY_ZERO] Division by zero. Use `try_divide` to tolerate divisor being 0 and return NULL instead. If necessary set "spark.sql.ansi...	ANSI mode again: division by zero now fails instead of returning null.	<pre># null when the employee was hired today try_divide(col("salary"), datediff(current_date(), col("hire_date")))</pre>
[NOT_ITERABLE] Column is not iterable.	Python's built-in sum(), min() or max() was applied to a Column.	<pre># sum imported as spark_sum (Setup #2) emp.agg(spark_sum("salary"))</pre>
[DATATYPE_MISMATCH.UNEXPECTED_INPUT_TYPE] Cannot resolve "datediff(salary, hire_date)" due to data type mismatch: The first parameter requires the "DATE" type, ho...	A function received the wrong column type, here an integer where a date is required.	<pre># both must be dates or timestamps datediff(col("updated_at"), col("hire_date"))</pre>

APPENDIX A · COMMON ERRORS

CONTINUED

ERROR	WHY IT HAPPENS	FIX
[PATH_NOT_FOUND] Path does not exist: employees.csv. SQLSTATE: 42K03	A typo, or the wrong root: Files/ on Fabric, /Volumes/ on Databricks.	<pre># Fabric notebookutils.fs.ls("Files/") # Databricks dbutils.fs .ls("/Volumes/main/hr/raw/")</pre>
[UNSUPPORTED_FEATURE. QUERY_ONLY_CORRUPT_RECORD_COLUMN] The feature is not supported: Queries from raw JSON/CSV/XML files are disallowed when the ...	Filtering or counting only _corrupt_record on a raw CSV or JSON read.	<pre>emp.cache() emp.filter(col("_corrupt_record") .isNotNull()).count()</pre>
No error, wrong data union() matches columns by position	On your data, a.union(b) with the columns in a different order put salaries into employee_id: 200 rows show an employee_id above 100,000.	<pre># matches by name: 0 such rows a.unionByName(b)</pre>

APPENDIX B · SPARK 3.5 VS 4.X

Spark 3.5 vs 4.x.

207

OF 209 RUN AS IS

Every open-source pattern (all but AI Functions and Notebook Utilities) was also run on Spark 3.5.9. Only two lines need a change, but five operations quietly return a different answer, because ANSI mode is off by default before Spark 4.

SAME CODE	SPARK 3.5	SPARK 4.X
<pre>col("employee_name").cast("int")</pre>	null	CAST_INVALID_INPUT
SAME ON BOTH	<pre>expr("try_cast(employee_name AS INT)")</pre>	
<pre>col("salary") / 0</pre>	null	DIVIDE_BY_ZERO
SAME ON BOTH	<pre>try_divide(col("salary"), lit(0))</pre>	

APPENDIX B · SPARK 3.5 VS 4.X

CONTINUED

	<pre>col("salary") * 100000 # salary 98,904</pre>	1,300,465,408: wrapped, wrong	ARITHMETIC_OVERFLOW
SAME ON BOTH	<pre>col("salary").cast("long") * 100000 # 9,890,400,000 on both</pre>		
	<pre>to_date(lit("2024-13-45"))</pre>	null	CAST_INVALID_INPUT
SAME ON BOTH	<pre>try_to_timestamp(lit("2024-13-45"), lit("yyyy-MM-dd"))</pre>		
	<pre>element_at(split(col("job_title"), " "), 9)</pre>	null	INVALID_ARRAY_INDEX_IN_ELEMENT_AT
SAME ON BOTH	<pre>try_element_at(split(col("job_title"), " "), lit(9))</pre>		

PATTERNS THAT NEED A NEWER SPARK

getAll: Spark 4.0+	See every setting	SECTION 02, PATTERN 05
Spark 4.0+	Cast a column to a new type	SECTION 06, PATTERN 01
Spark 3.4+	Filter in SQL with parameters	SECTION 08, PATTERN 12
Spark 3.4+	Rename multiple columns	SECTION 10, PATTERN 04
Spark 3.3+	Same change to many columns	SECTION 14, PATTERN 01
Spark 3.3+	Trigger modes	SECTION 24, PATTERN 08
Spark 3.5+	Unit-test a transformation	SECTION 29, PATTERN 09

On Spark 3.5, write `expr("try_cast(col AS INT)")` instead of `.try_cast("int")`, and `spark.sql("SET")` instead of `spark.conf.getAll`. Every Delta badge in this sheet is covered by Delta 3.2, which Fabric Runtime 1.3 ships.

APPENDIX B • SPARK 3.5 VS 4.X

CONTINUED

MAKE SPARK 3.5 BEHAVE LIKE 4

```
spark.conf
.set("spark.sql.ansi.enabled",
    "true")
```

Checked on 3.5.9: the cast and overflow rows above then raise the same errors as 4.0.

WHICH SPARK IS MY RUNTIME?

Fabric Runtime 1.3	Spark 3.5
Fabric Runtime 2.0	Spark 4.1
Databricks 15.4 LTS, 16.4 LTS	Spark 3.5
Databricks 17.3 LTS	Spark 4.0
Databricks 18 LTS	Spark 4.1

New Fabric workspaces still default to Runtime 1.3, which has an end-of-support date announced. This sheet was tested on 3.5 and 4.0; check 4.1 runtimes against this table.

CHANGE NOTES

What changed since 3.0.

23 to 29
SECTIONS

193 to 221
PATTERNS

Compared with the 3.0 edition: what is new, what changed, what went, and what 3.0 got wrong.

NEW

7 new sections	Config Settings, Notebook Utilities, Data Types, Loops, AI Functions, SCD Type 1, Spark Plans
10 production patterns	replaceWhere, dynamic partition overwrite, mergeSchema on read and on write, Change Data Feed, CHECK constraints, foreachBatch MERGE, session time zone, parameterized <code>spark.sql</code> , <code>assertDataFrameEqual</code>
Deep dives	Joins and Window Functions with row-by-row results from the sample data
Platform panels	Databricks and Microsoft Fabric notes in 9 sections
Version badges	minimum Spark or Delta version on 12 patterns
Appendices	A: common errors and tested fixes. B: Spark 3.5 vs 4.x, run on both
Companion notebook	runs every pattern and the Delta claims in your Fabric or Databricks workspace

CHANGE NOTES · WHAT CHANGED SINCE 3.0

CONTINUED

CHANGED

Structure	five chapters named by task; sections reordered so each builds on the one before; a divider page per chapter
One dataset	every example uses employees.csv and departments.csv
Imports	the <code>F.</code> prefix replaced by named imports; sum, min and max aliased
Watch-outs	trimmed from 100 to 54
Layout	12 section layouts, code at 9.5pt, and the font embedded so pages look the same offline

REMOVED

Sections	I/O: Cloud Storage: used less often
Duplicates	14 patterns that repeated another; each kept in the section where it fits best

CORRECTED: WRONG IN 3.0

startsWith	<code>startsWith</code> and <code>endsWith</code> are <code>startswith</code> and <code>endswith</code> in PySpark
MetadataBuilder	does not exist in PySpark: metadata is a dict passed to <code>alias()</code>
vacuum	<code>DeltaTable.vacuum()</code> has no <code>dry_run</code> : use SQL <code>VACUUM ... DRY RUN</code>
MERGE options	MERGE has no <code>.option()</code> : schema evolution is <code>.withSchemaEvolution()</code> , Delta 3.2+
NOT MATCHED BY SOURCE	needs Delta 2.3+, not 2.0
Streaming sink	a streaming write had no output format
Sorting	ascending sorts put nulls first, not last
dropDuplicates	does not promise to keep the first occurrence
Casts	with ANSI on, the Spark 4 default, a bad cast raises an error instead of returning null
Conditional MERGE	<code>!=</code> skipped changes to or from null: now <code><=></code>
spark.stop()	3.0 said always stop the session, with no warning about notebooks: now scripts and jobs only



LAST WORD

Read the plan before you tune anything.

Across 29 sections and 221 patterns the same habits repeat: define the schema, filter early, broadcast the small side, and validate at every stage boundary. The fastest fix is usually the one already written down.

Sanjay Chandra

The Databricks + Fabric guy on LinkedIn



A NOTE ON THIS RELEASE

Version 4.0. Open-source PySpark and Delta Lake, tested on Spark 3.5 and 4.0, plus Databricks and Fabric features where marked. Spotted an inaccuracy? Feedback is genuinely welcome.

ACKNOWLEDGEMENTS

Claude (Anthropic) was used as a writing assistant while putting this cheat sheet together.

CONNECT

www.ssanjaychandra.com
linkedin.com/in/ssanjaychandra
ssanjaychandra@outlook.com